

Laporan Akhir Tugas Besar
Pemrograman Berorientasi Objek
PlanWise



Hakim Akram Azizi	103012400009
Azh Zhahir Ramadhan Putra	103012400223
Muhammad Ihsan Abdurrasyid	103012400354
Muhammad Rifki Widya R	103012400186
Muhammad Sylva Sadjandegi	103012400132

S1 Informatika
Fakultas Informatika
Universitas Telkom
2026

DAFTAR ISI

Daftar Isi.....	1
DAFTAR GAMBAR.....	4
1 PENDAHULUAN.....	5
1.1 LATAR BELAKANG.....	5
1.2 TUJUAN.....	5
1.3 DESKRIPSI UMUM SISTEM (RUANG LINGKUP).....	5
Ruang Lingkup Sistem.....	5
1.3.1 Fungsionalitas Utama Sistem.....	6
1.3.2 Batasan Teknis dan Sistem (Constraints).....	6
1.4 METODOLOGI PENGEMBANGAN SISTEM.....	7
1.4.1 ANALISIS KEBUTUHAN.....	7
1.4.2 TAHAPAN PERANCANGAN.....	7
1.4.3 IMPLEMENTASI KODE.....	7
2 DASAR TEORI.....	8
2.1 Pemrograman Berorientasi Objek.....	8
2.2 Unified Modeling Language (UML).....	8
2.3 Java.....	8
2.4 PostgreSQL.....	8
2.5 Servlet/JSP/Springboot.....	8
3 ANALISIS DAN IMPLEMENTASI SISTEM.....	9
3.1 ANALISIS KEBUTUHAN SISTEM.....	9
3.1.1 MODUL AUTENTIKASI & PROFIL PENGGUNA.....	9
3.1.2 Modul Manajemen Papan Tugas (CRUD Task).....	9
3.1.3 Modul Pelacakan Status & Kanban Board.....	9
3.1.4 Modul Label, Prioritas, Filter & Sort.....	9
3.1.5 Modul Riwayat Aktivitas & Notifikasi.....	10
3.2.1.2. Entitas Tasks.....	13
Entitas Tasks merupakan entitas utama yang menyimpan informasi tugas.....	13
3.2.1.3. Entitas Management_Tasks.....	13
3.2.1.4. Entitas Group_Task_Members.....	14
Entitas Group_Task_Members digunakan untuk menyimpan data anggota dalam tugas kelompok.....	14
3.2.1.5. Entitas Group_Task.....	14
3.2.1.6. Entitas Group_Task_Members.....	14
5.3 Fitur Manajemen Task.....	37
5.3.1 Kasus Uji.....	37

DAFTAR TABEL

Tabel 1 Tabel Entitas User.....	17
Tabel 2 Tabel Entitas task.....	18
Tabel 3 Tabel Entitas management task.....	19
Tabel 3 Tabel Entitas Labels.....	19
Tabel 4 Tabel Entitas Task Labels.....	19
Tabel 5 Tabel Entitas histories.....	20
Tabel 6 Tabel Notofications.....	20
Tabel 7 Tabel Personal task.....	21
Tabel 8 Tabel Deadline task.....	21
Tabel 9 Tabel Group task.....	22
Tabel 10 Tabel Group task member.....	22
Tabel 2 Identifikasi Pengujian.....	35
Tabel 3 Kasus Uji Fitur Login.....	37
Tabel 4 Hasil Uji Fitur Login.....	37
Tabel 7 Kasus Uji Fitur Manajemen Task.....	37
Tabel 6 Hasil Uji Fitur Manajemen Task.....	38
Tabel 7 Kasus Uji Fitur Label & Filter.....	38
Tabel 8 Hasil Uji Fitur Label & Filter.....	39
Tabel 9 Kasus Uji Fitur History.....	39
Tabel 10 Hasil Uji Fitur History.....	40

Daftar Gambar

Gambar 2 class diagram.....	23
-----------------------------	----

1 Pendahuluan

1.1 Latar Belakang

Di era digital saat ini, mahasiswa dihadapkan pada berbagai tugas akademik, proyek kelompok, dan jadwal kegiatan yang perlu dikelola secara terstruktur. Tanpa sistem manajemen yang baik, deadline dapat terlewat, progres tugas sulit dipantau, dan pembagian pekerjaan kelompok menjadi tidak merata.

PlanWise dirancang sebagai aplikasi manajemen tugas berbasis web yang sederhana, ringan, dan sesuai dengan kebutuhan mahasiswa. Tampilan utama aplikasi menggunakan task board sehingga pengguna dapat melihat daftar tugas berdasarkan status seperti To Do, In Progress, Review, dan Done. Selain itu, pengguna dapat melakukan pencarian, filter, sort, pemberian label, pengaturan prioritas, serta melihat notifikasi yang berkaitan dengan tugas.

Dari sisi perancangan berorientasi objek, PlanWise dibangun dengan class utama Task, User, ManagementTask, Label, History, dan Notification. Struktur class juga memanfaatkan inheritance melalui PersonalTask, GroupTask, dan DeadlineTask, serta interface Trackable dan Filterable untuk mendukung pelacakan status dan penyaringan data task.

1.2 Tujuan

- Membangun aplikasi manajemen tugas berbasis web yang mudah digunakan oleh mahasiswa.
- Mengimplementasikan konsep Pemrograman Berorientasi Objek, meliputi class, object, encapsulation, inheritance, polymorphism, dan interface.
- Menyediakan fitur task board untuk membuat, melihat, memperbarui, menghapus, dan memantau tugas.
- Menyediakan fitur label, prioritas, filter, sort, status tracking, riwayat aktivitas, dan notifikasi in-app.
- Menyelaraskan rancangan fitur, class diagram, dan mockup UI agar implementasi aplikasi lebih konsisten.

1.3 Deskripsi Umum Sistem (Ruang Lingkup)

Deskripsi Umum

PlanWise adalah aplikasi manajemen tugas (*task management*) berbasis web yang dirancang khusus untuk membantu mahasiswa dalam mengelola beban kerja akademik, proyek kelompok, dan jadwal kegiatan secara terstruktur. Aplikasi ini mengadopsi pendekatan visual menggunakan papan tugas (*task board* / Kanban board) guna mempermudah pemantauan progres tugas secara *real-time*.

Sistem ini memfasilitasi kolaborasi dan pengorganisasian tugas melalui pembagian jenis tugas yang fleksibel, pelacakan status yang transparan, serta sistem penyaringan data yang adaptif. Diperkuat dengan konsep Pemrograman Berorientasi Objek (OOP), PlanWise mengintegrasikan manajemen data pengguna, pelabelan, riwayat aktivitas, dan notifikasi internal di dalam satu platform yang ringan dan mudah digunakan.

Ruang Lingkup Sistem

Ruang lingkup fungsionalitas dan teknis dari pengembangan sistem PlanWise dibatasi pada poin-poin berikut:

1.3.1 Fungsionalitas Utama Sistem

- **Autentikasi & Profil Pengguna:** Mengelola proses registrasi, login, dan logout pengguna, serta menyajikan informasi profil aktif berdasarkan peran (*role*) masing-masing pada antarmuka.
- **Manajemen Papan Tugas (CRUD Task):** Pengguna dapat membuat, melihat, memperbarui, dan menghapus tugas. Tugas dikategorikan menjadi tiga jenis objek spesifik: *Personal Task*, *Group Task*, dan *Deadline Task*.
- **Pelacakan Status & Kanban Board:** Visualisasi alur kerja tugas yang dibagi ke dalam 4 tahapan status utama, yaitu: To Do, In Progress, Review, dan Done. Perubahan status dilakukan untuk memantau perkembangan tugas secara visual.
- **Label, Prioritas, & Manajemen Pencarian:** Menyediakan fitur penambahan label/kategori, pengaturan tingkat prioritas tugas (*Low*, *Medium*, *High*), serta kemampuan pemfilteran (*filtering*) dan pengurutan (*sorting*) untuk mempermudah pencarian tugas.
- **Riwayat Aktivitas & Notifikasi:** Sistem secara otomatis mencatat setiap perubahan penting pada tugas (seperti pembuatan tugas, perubahan status, atau modifikasi tenggat waktu) ke dalam log riwayat, sekaligus mengirimkan notifikasi langsung di dalam aplikasi (*in-app notification*).

1.3.2 Batasan Teknis dan Sistem (Constraints)

- **Platform:** Aplikasi ini dikembangkan murni berbasis web (*web-based application*) menggunakan framework Next.js dan Tailwind CSS, serta tidak mencakup pengembangan aplikasi *mobile native* (Android/iOS).
- **Arsitektur Kode:** *Backend* aplikasi atau konsep berorientasi objeknya dirancang menggunakan bahasa pemrograman Java, sedangkan sisi *frontend* memanfaatkan JavaScript/TypeScript.
- **Mekanisme Notifikasi:** Notifikasi bersifat *in-app* (hanya muncul saat pengguna membuka aplikasi) dan tidak terintegrasi dengan layanan eksternal seperti email blasting, SMS Gateway, atau *push notification* pihak ketiga.
- **Penyimpanan Data (Prototipe):** Untuk tahap perancangan dan prototipe saat ini, penyimpanan data masih menggunakan *local storage*, file lokal, atau *in-memory storage* (belum diwajibkan menggunakan database server eksternal yang kompleks).
- **Komunikasi:** Sistem tidak menyediakan fitur obrolan langsung (*real-time chat*) antar-pengguna di dalam platform.

1.4 Metodologi Pengembangan Sistem

1.4.1 Analisis Kebutuhan

- **Analisis Masalah:** Mengidentifikasi masalah manajemen tugas pada mahasiswa yang melatarbelakangi pembuatan aplikasi.
- **Pendefinisian Spesifikasi:** Menentukan batasan teknologi yang digunakan seperti bahasa pemrograman Java, JavaScript/TypeScript, Next.js, dan Tailwind CSS.

1.4.2 Tahapan Perancangan

- **Perancangan Berorientasi Objek:** Menyusun struktur *Class Diagram* (mencakup *Task*, *User*, *History*, *Notification*, *ManagementTask*, dll.) serta menentukan relasi *inheritance* dan *interface*.
- **Perancangan Antarmuka (UI/UX):** Membuat *wireframe* dan *mockup* visual untuk halaman utama *Task Board* dan halaman *Authentication* sebagai acuan visual implementasi *frontend*.

1.4.3 Implementasi Kode

- **Iterasi 1 (Autentikasi):** Implementasi *class* *User* untuk menangani registrasi, login, logout, dan manajemen sesi profil.
- **Iterasi 2 (Manajemen Task):** Implementasi fungsi CRUD pada *class* *Management Task* beserta sub-class *PersonalTask*, *GroupTask*, dan *DeadlineTask*.
- **Iterasi 3 (Tracking & Board):** Implementasi *interface* *Trackable* dan visualisasi perpindahan status tugas pada kolom *Kanban*.
- **Iterasi 4 (Filter & Sort):** Implementasi logika penyaringan data berdasarkan label, tingkat prioritas (*Low*, *Medium*, *High*), dan pengurutan objek *Task*.
- **Iterasi 5 (History & Notifikasi):** Implementasi pencatatan log aktivitas lewat *class* *History* dan pemicu *in-app notification* lewat *class*

1.4.4 Integrasi dan Pengujian

- **Integrasi Frontend & Backend:** Menghubungkan visual antarmuka Next.js/Tailwind dengan logika berorientasi objek yang telah dibangun.
- **Pengujian Fungsional :** Memastikan semua metode dalam *class diagram* (seperti *updateStatus()*, *checkOverdue()*, *filterByLabel()*) berjalan sesuai dengan ekspektasi dan tidak menyebabkan bentrok.

2 Dasar Teori

2.1 Pemrograman Berorientasi Objek

Pemrograman Berorientasi Objek (OOP) adalah paradigma pemrograman yang mengorganisasikan perangkat lunak dalam bentuk kumpulan objek yang saling berinteraksi. Setiap objek merupakan instansi dari sebuah kelas (*class*) dan memiliki atribut (data) serta perilaku (method). Konsep utama OOP yang diterapkan dalam PlanWise meliputi:

- Encapsulation: Atribut kelas disembunyikan (akses private) dan diakses melalui getter/setter, diterapkan pada seluruh entity seperti Task, User, dan Label.
- Inheritance: PersonalTask, GroupTask, dan DeadlineTask mewarisi class Task, sehingga seluruh properti dan method task dasar dapat digunakan kembali.
- Polymorphism: Method getType() di-override pada setiap subclass (PersonalTask → "PERSONAL", GroupTask → "GROUP", DeadlineTask → "DEADLINE").
- Interface: Trackable mendefinisikan kontrak pelacakan status; Filterable mendefinisikan kontrak penyaringan data task.

2.2 Unified Modeling Language (UML)

UML adalah bahasa pemodelan standar yang digunakan untuk memvisualisasikan, merancang, dan mendokumentasikan sistem perangkat lunak. Dalam PlanWise, diagram yang digunakan adalah Class Diagram, yang menggambarkan struktur kelas, atribut, method, relasi inheritance, dan interface. Diagram dibuat menggunakan draw.io.

2.3 Java

Java adalah bahasa pemrograman berorientasi objek yang berjalan di atas Java Virtual Machine (JVM). Dalam PlanWise, Java digunakan sebagai bahasa backend melalui framework Spring Boot. Versi yang digunakan adalah Java 17+ (sesuai kebutuhan Spring Boot 3.x). Fitur Java yang dimanfaatkan antara lain generics (List<Task>), stream API untuk filtering/sorting, annotation JPA (@Entity, @OneToMany, @ManyToMany), dan Lombok untuk mengurangi boilerplate.

2.4 PostgreSQL

Basis data yang digunakan adalah PostgreSQL yang di-host melalui layanan Supabase. PostgreSQL adalah sistem manajemen basis data relasional (RDBMS) open-source yang mendukung ACID, JSON, dan query kompleks. Akses database dari Java dilakukan melalui Spring Data JPA dengan driver PostgreSQL JDBC. Supabase dipilih karena menyediakan cloud PostgreSQL dengan konfigurasi minimal.

2.5 Servlet/JSP/Springboot

PlanWise menggunakan Spring Boot sebagai framework backend, menggantikan pendekatan Servlet/JSP tradisional. Spring Boot menyediakan auto-configuration, embedded server (Tomcat), dan ekosistem Spring yang lengkap. Lapisan yang digunakan:

- *Spring Web MVC (REST Controller) untuk menangani HTTP request/response*
- *Spring Data JPA untuk abstraksi akses database*
- *Lombok untuk mengurangi kode boilerplate*

Frontend dikembangkan dengan React + Vite (bukan JSP), yang berkomunikasi ke backend melalui REST API menggunakan Axios.

3 Analisis dan Implementasi Sistem

3.1 Analisis Kebutuhan Sistem

3.1.1 Modul Autentikasi & Profil Pengguna

- Sistem harus dapat melakukan registrasi akun baru dengan meminta data berupa nama, email, password, dan peran (role) pengguna.
- Sistem harus dapat memvalidasi data login dan menangani proses logout untuk mengamankan sesi (session) pengguna yang aktif.
- Sistem harus dapat menampilkan informasi profil pengguna yang sedang aktif (berupa avatar, nama, dan role) pada bagian kanan atas halaman utama (header).

3.1.2 Modul Manajemen Papan Tugas (CRUD Task)

- Sistem harus dapat membuat tugas (Create Task) baru melalui tombol "Add Task" pada kolom papan tugas.
- Sistem harus dapat menampilkan daftar tugas (Read Task) secara terstruktur pada papan tugas berdasarkan statusnya.
- Sistem harus dapat memperbarui data tugas (Update Task), meliputi perubahan judul, deskripsi, tenggat waktu (deadline), prioritas, label, dan status.
- Sistem harus dapat menghapus tugas (Delete Task) yang sudah tidak diperlukan dari papan tugas.
- Sistem harus dapat mengelompokkan tugas ke dalam tiga tipe objek spesifik: Personal Task, Group Task, dan Deadline Task.

3.1.3 Modul Pelacakan Status & Kanban Board

- Sistem harus dapat memvisualisasikan tugas ke dalam papan Kanban yang terdiri dari 4 kolom status utama: To Do, In Progress, Review, dan Done.
- Sistem harus dapat mengubah status progres tugas menggunakan fungsi internal sistem (updateStatus dan changeStatus).
- Sistem harus dapat melakukan pengecekan otomatis terhadap tugas yang memiliki tenggat waktu (overdue check) melalui objek Deadline Task.

3.1.4 Modul Label, Prioritas, Filter & Sort

- Sistem harus dapat menambahkan dan menghapus label atau kategori penanda pada setiap tugas.

- Sistem harus dapat menetapkan tingkat prioritas pada tugas dengan kategori Low, Medium, dan High.
- Sistem harus dapat menyaring (filter) daftar tugas yang tampil berdasarkan label, tingkat prioritas, maupun status tertentu.
- Sistem harus dapat mengurutkan (sort) tugas di papan Kanban agar pengguna lebih mudah mendahului tugas yang mendesak.

3.1.5 Modul Riwayat Aktivitas & Notifikasi

- Sistem harus dapat mencatat setiap log aktivitas penting (seperti membuat tugas, mengubah status, mengganti deadline, atau menambah label) ke dalam sistem riwayat.
- Sistem harus dapat menampilkan dan mengurutkan riwayat aktivitas berdasarkan tugas spesifik tertentu.
- Sistem harus dapat memicu dan mengirimkan notifikasi di dalam aplikasi (in-app notification) kepada pengguna ketika terjadi perubahan status atau deadline tugas, serta menandai notifikasi tersebut jika sudah dibaca.

3.1.6 Kebutuhan Non-Fungsional

- **Ketersediaan (*Availability*):** Sistem harus dapat diakses oleh mahasiswa kapan saja (asumsi ideal operasional lokal/server) untuk memantau tugas harian.
- **Kemudahan Penggunaan (*Usability*):** Antarmuka aplikasi harus dirancang sederhana, ringan, menggunakan pendekatan visual *task board* agar mudah dipahami oleh mahasiswa awam sekalipun.
- **Batasan Lingkup (*Constraints*):** * Aplikasi hanya berjalan pada platform berbasis web dan tidak menyediakan versi aplikasi *mobile native*.
 - Sistem tidak menyediakan ruang obrolan langsung (*real-time chat*) antar anggota kelompok.
 - Sistem notifikasi dibatasi hanya muncul di dalam aplikasi (*in-app*), bukan melalui SMS, Email, atau *push notification* eksternal.

3.1.6.1 Spesifikasi Perangkat Lunak

- **Bahasa Pemrograman:** Java (untuk pemodelan konsep logika/backend berorientasi objek) dan JavaScript/TypeScript (untuk pengembangan antarmuka pengguna).
- **Framework & Library:** Next.js (framework aplikasi web) dan Tailwind CSS (framework desain antarmuka).
- **Lingkungan Pengembangan (IDE):** Visual Studio Code.
- **Manajemen Versi (VCS):** Git dan layanan repositori GitHub.
- **Alat Pemodelan (Diagramming Tools):** draw.io (untuk penyusunan *Class Diagram*).
- **Sistem Operasi Pengembangan:** Windows 10 atau Windows 11.

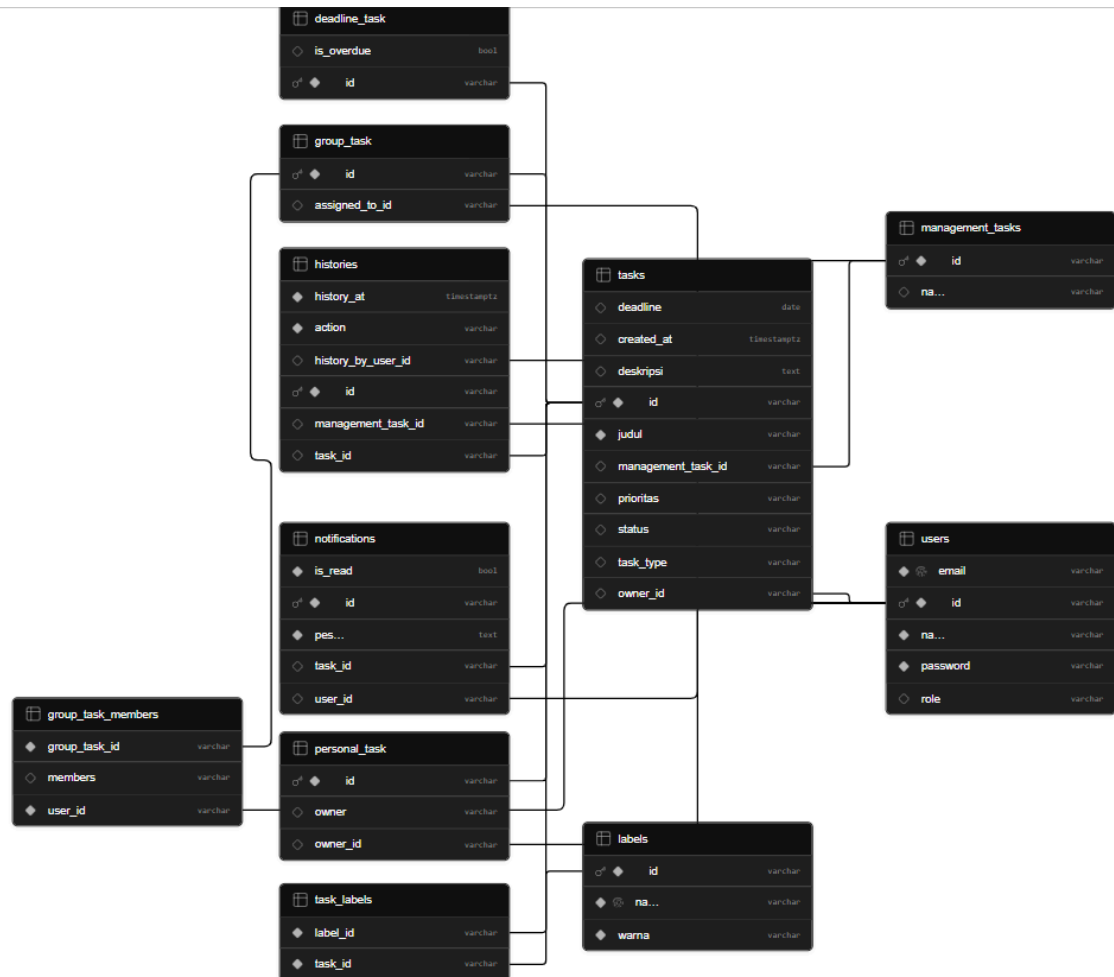
3.1.6.2. Spesifikasi Perangkat Keras

- **Prosesor (CPU):** Intel Core i3 / AMD Ryzen 3 (Generasi modern yang mendukung kelancaran *runtime* Node.js dan kompilasi Java).
- **Memori (RAM):** Minimal 4 GB atau 8 GB (Direkomendasikan 8 GB agar eksekusi IDE Visual Studio Code dan *local development server* Next.js tidak mengalami hambatan).
- **Penyimpanan (Storage):** Tersedia ruang kosong minimal 2 GB pada HDD/SSD untuk menyimpan repositori kode proyek, dependensi *node_modules*, dan berkas log riwayat lokal.
- **Perangkat Input/Output:** Monitor dengan resolusi standar (minimal 1366x768) agar tampilan visual dari *Kanban Board* dapat dipantau dengan jelas.

3.2 Implementasi Basis Data

3.2.1 Entity Relationship Diagram (ERD)

Menampilkan dan menjelaskan diagram ERD



Gambar 1 Tabel ERD

3.2.1.1. Entitas Users

Entitas **Users** digunakan untuk menyimpan data pengguna yang dapat mengakses sistem.

Atribut:

- id (PK) : identitas unik pengguna.
- nama : nama pengguna.
- email : alamat email pengguna.
- password : kata sandi pengguna.
- role : peran pengguna dalam sistem (admin, user, dll).

Relasi:

- Berelasi dengan entitas **Tasks** melalui atribut owner_id.
- Satu pengguna dapat memiliki banyak tugas (**One-to-Many**).
- Berelasi dengan **Notifications** melalui user_id.
- Berelasi dengan **Group_Task_Members** melalui user_id.

3.2.1.2. Entitas Tasks

Entitas **Tasks** merupakan entitas utama yang menyimpan informasi tugas.

Atribut:

- id (PK) : identitas unik tugas.
- judul : judul tugas.
- deskripsi : deskripsi tugas.
- deadline : batas waktu penyelesaian tugas.
- created_at : waktu pembuatan tugas.
- prioritas : tingkat prioritas tugas.
- status : status tugas.
- task_type : jenis tugas.
- owner_id (FK) : pemilik tugas.
- management_task_id (FK) : kategori/manajemen tugas.

Relasi:

- Banyak tugas dimiliki oleh satu **Users** (**Many-to-One**).
- Banyak tugas dapat berada dalam satu **Management_Tasks** (**Many-to-One**).
- Berelasi dengan **Histories**, **Notifications**, dan **Task_Labels**.
- Berelasi dengan **Personal_Task**, **Group_Task**, dan **Deadline_Task** sebagai turunan/jenis tugas.

3.2.1.3. Entitas Management_Tasks

Entitas **Management_Tasks** digunakan untuk mengelompokkan atau mengategorikan tugas.

Atribut:

- id (PK) : identitas kategori tugas.
- name : nama kategori/manajemen tugas.

Relasi:

- Satu kategori dapat memiliki banyak **Tasks (One-to-Many)**.
- Berelasi dengan **Histories** melalui management_task_id.

3.2.1.4. Entitas Group_Task_Members

Entitas **Group_Task_Members** digunakan untuk menyimpan data anggota dalam tugas kelompok.

Atribut:

- group_task_id (FK) : identitas tugas kelompok.
- user_id (FK) : identitas pengguna.
- members : informasi anggota.

Relasi:

- Berelasi dengan **Group_Task** melalui group_task_id.
- Berelasi dengan **Users** melalui user_id.
- Membentuk hubungan **Many-to-Many** antara **Group_Task** dan **Users**.

3.2.1.5. Entitas Group_Task

Entitas **Group_Task** digunakan untuk menyimpan tugas yang dikerjakan secara kelompok.

Atribut:

- id (PK/FK) : identitas tugas kelompok.
- assigned_to_id : pengguna yang ditugaskan.

Relasi:

- Berelasi dengan **Tasks** melalui id.
- Berelasi dengan **Group_Task_Members** untuk menyimpan anggota kelompok.
- Satu tugas kelompok dapat memiliki banyak anggota (**One-to-Many**).

3.2.1.6. Entitas Group_Task_Members

Entitas **Group_Task_Members** digunakan untuk menyimpan data anggota dalam tugas kelompok.

Atribut:

- group_task_id (FK) : identitas tugas kelompok.
- user_id (FK) : identitas pengguna.
- members : informasi anggota.

Relasi:

- Berelasi dengan **Group_Task** melalui group_task_id.
- Berelasi dengan **Users** melalui user_id.
- Membentuk hubungan **Many-to-Many** antara **Group_Task** dan **Users**.

3.2.1.7. Entitas Deadline_Task

Entitas **Deadline_Task** digunakan untuk menyimpan informasi keterlambatan tugas.

Atribut:

- id (PK/FK) : identitas tugas.
- is_overdue : status apakah tugas telah melewati deadline.

Relasi:

- Berelasi dengan **Tasks** melalui id.
- Satu tugas memiliki satu status deadline (**One-to-One**).

3.2.1.8. Entitas Histories

Entitas **Histories** digunakan untuk mencatat riwayat aktivitas pada sistem.

Atribut:

- id (PK) : identitas riwayat.
- action : aktivitas yang dilakukan.
- history_at : waktu aktivitas.
- history_by_user_id : pengguna yang melakukan aktivitas.
- task_id (FK) : tugas yang terkait.
- management_task_id (FK) : kategori tugas yang terkait.

Relasi:

- Banyak riwayat dapat dimiliki oleh satu **Tasks** (**Many-to-One**).
- Banyak riwayat dapat terkait dengan satu **Management_Tasks** (**Many-to-One**).

3.2.1.9. Entitas Histories

Entitas **Histories** digunakan untuk mencatat riwayat aktivitas pada sistem.

Atribut:

- **id** (PK) : identitas riwayat.
- **action** : aktivitas yang dilakukan.
- **history_at** : waktu aktivitas.
- **history_by_user_id** : pengguna yang melakukan aktivitas.
- **task_id** (FK) : tugas yang terkait.
- **management_task_id** (FK) : kategori tugas yang terkait.

Relasi:

- Banyak riwayat dapat dimiliki oleh satu **Tasks** (**Many-to-One**).
- Banyak riwayat dapat terkait dengan satu **Management_Tasks** (**Many-to-One**).

3.2.1.10. Entitas Labels

Entitas **Labels** digunakan untuk memberi tag atau penanda pada tugas.

Atribut:

- **id** (PK) : identitas label.
- **nama** : nama label.
- **warna** : warna label.

Relasi:

- Berelasi dengan **Tasks** melalui entitas **Task_Labels**.
- Satu label dapat digunakan oleh banyak tugas (**Many-to-Many**).

3.2.1.11. Entitas Task_Labels

Entitas **Task_Labels** merupakan tabel penghubung antara tugas dan label.

Atribut:

- **task_id** (FK) : identitas tugas.
- **label_id** (FK) : identitas label.

Relasi:

- Berelasi dengan **Tasks** melalui **task_id**.
- Berelasi dengan **Labels** melalui **label_id**.
- Membentuk hubungan **Many-to-Many** antara **Tasks** dan **Labels**.

3.2.2 Gambar 1 ERD Sistem

3.2.3 Struktur Basis data

3.2.3.1. Tabel User

Tabel *User* dirancang sebagai berikut;

Tabel 1 Tabel Entitas User

<i>Field</i>	<i>Tipe</i>	<i>Key</i>
id	<i>VARCHAR</i>	<i>Primary Key</i>
email	<i>VARCHAR</i>	<i>Unique Key</i>
nama	<i>VARCHAR</i>	-
password	<i>VARCHAR</i>	-
role	<i>VARCHAR</i>	-

3.2.3.2. Tabel Tasks

Tabel *Tasks* dirancang sebagai berikut;

Tabel 2 Tabel Entitas task

Field	Tipe	Key
id	VARCHAR	Primary Key
judul	VARCHAR	-
deskripsi	TEXT	-
deadline	DATE	-
created_at	TIMESTAMPTZ	-
status	VARCHAR	-
prioritas	VARCHAR	-
task_type	VARCHAR	-
owner_id	VARCHAR	Foreign Key
management_task_id	VARCHAR	Foreign Key

3.2.3.3. Tabel Management Task

Tabel *Management Tasks* dirancang sebagai berikut;

Tabel 3 Tabel Entitas management task

Field	Tipe	Key
id	VARCHAR	Primary Key
nama	VARCHAR	-

3.2.3.4. Tabel labels & Task Labels

Tabel *Labels* dirancang sebagai berikut;

Tabel 3 Tabel Entitas Labels

Field	Tipe	Key
id	VARCHAR	Primary Key
nama	VARCHAR	Unique Key
warna	VARCHAR	-

Tabel 4 Tabel Entitas Task Labels

Field	Tipe	Key
-------	------	-----

label_id	VARCHAR	Foreign Key
task_id	VARCHAR	Foreign Key

3.2.3.5. Tabel Histories

Tabel *Histories* dirancang sebagai berikut;

Tabel 5 Tabel Entitas histories

<i>Field</i>	<i>Tipe</i>	<i>Key</i>
id	<i>VARCHAR</i>	<i>Primary Key</i>
action	<i>VARCHAR</i>	-
history_at	<i>TIMESTAMPTZ</i>	-
history_by_user_id	<i>VARCHAR</i>	<i>Foreign Key</i>
task_id	<i>VARCHAR</i>	<i>Foreign Key</i>
management_task_id	<i>VARCHAR</i>	<i>Foreign Key</i>

3.2.3.6. Tabel Notification

Tabel *Notifications* dirancang sebagai berikut;

Tabel 6 Tabel Notofications

<i>Field</i>	<i>Tipe</i>	<i>Key</i>

id	<i>VARCHAR</i>	<i>Primary Key</i>
pesan	<i>TEXT</i>	-
is_read	<i>BOOL</i>	-
task_id	<i>VARCHAR</i>	<i>Foreign Key</i>
user_id	<i>VARCHAR</i>	<i>Foreign Key</i>

3.2.3.7. Tabel Inheritance Tugas
Tabel *Tugas* dirancang sebagai berikut;
Tabel 7 Tabel Personal task

Field	Tipe	Key
id	VARCHAR	Primary Key
owner	VARCHAR	-
owner_id	VARCHAR	-

Tabel 8 Tabel Deadline task

<i>Field</i>	<i>Tipe</i>	<i>Key</i>

id	<i>VARCHAR</i>	<i>Primary Key</i>
is_overdue	<i>BOOL</i>	-

Tabel 9 Tabel Group task

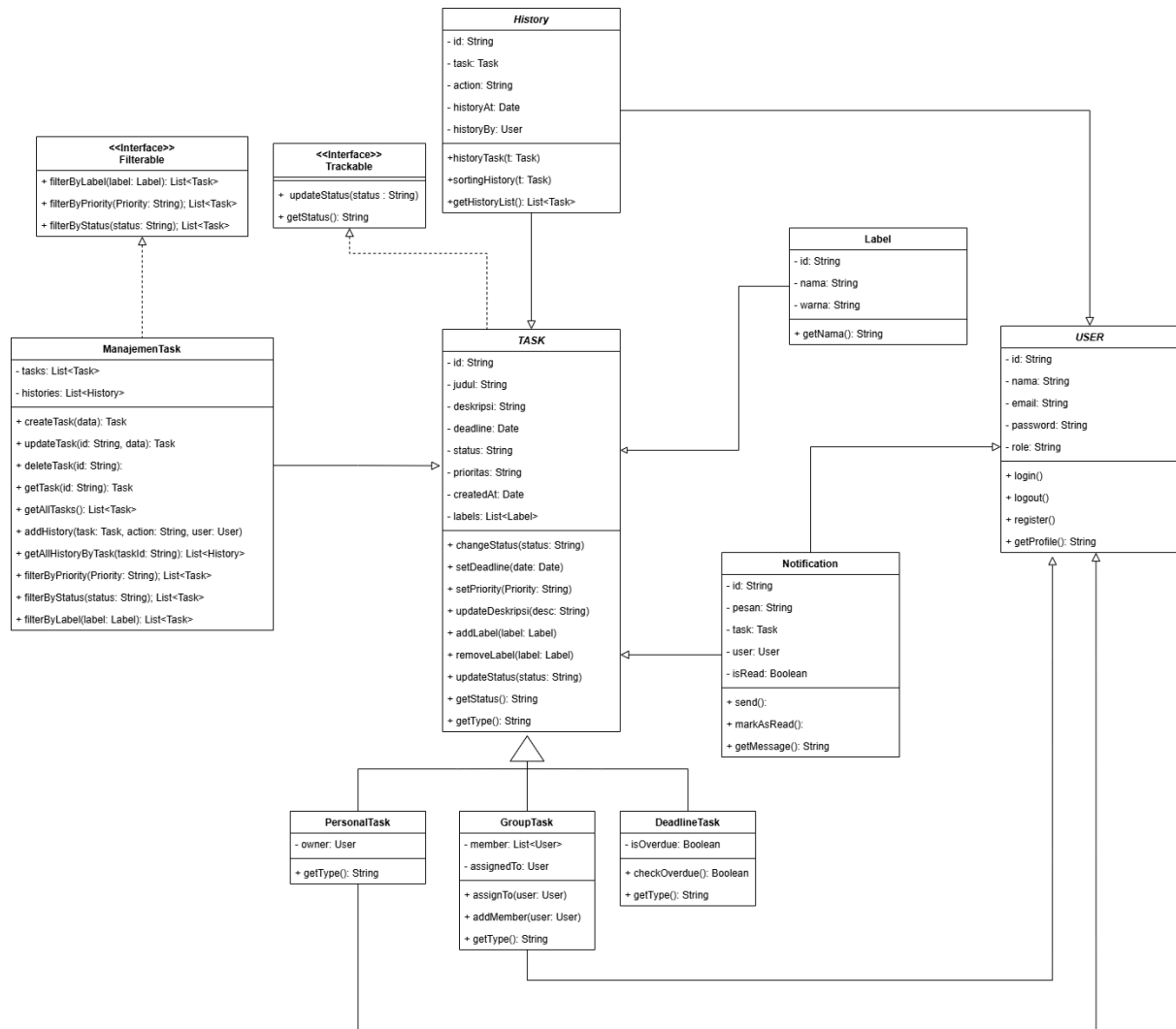
Field	Tipe	Key
id	VARCHAR	Primary Key
assigned_to_id	VARCHAR	-

Tabel 10 Tabel Group task member

Field	Tipe	Key
group_task_id	VARCHAR	Foreign Key
members	VARCHAR	-
user_id	VARCHAR	Foreign Key

3.3 Implementasi UML

3.3.1 *Class Diagram*



Gambar 2 class diagram

3.3.2 Deskripsi Kelas

3.3.2.1. Kelas Filterable (Interface)

Antarmuka yang mendefinisikan kontrak standar untuk fungsionalitas penyaringan (*filtering*) data objek tugas.

- Metode (*Methods*):
 - + **filterByLabel(label: Label): List<Task>**: Menyaring dan mengembalikan daftar objek tugas berdasarkan komponen label tertentu.
 - + **filterByPriority(priority: String): List<Task>**: Menyaring dan mengembalikan daftar objek tugas berdasarkan tingkat prioritas (*Low/Medium/High*).
 - + **filterByStatus(status: String): List<Task>**: Menyaring dan mengembalikan daftar objek tugas berdasarkan status progresnya.

3.3.2.2. Kelas Trackable (Interface)

Antarmuka yang mendefinisikan standar pelacakan dan perubahan status perkembangan dari sebuah objek tugas.

- Metode (*Methods*):
 - + **updateStatus(status: String): void**: Mengubah status berjalan dari suatu objek ke status yang baru.
 - + **getStatus(): String**: Mengambil nilai teks status operasional terkini dari objek.

3.3.2.3. Kelas Manajemen Task

Kelas kontroler utama aplikasi yang mengimplementasikan *interface* Filterable untuk mengelola siklus hidup data tugas dan log riwayat.

- Atribut (*Attributes*):
 - - **tasks: List<Task>**: Menyimpan daftar seluruh objek tugas yang terdaftar di dalam sistem.
 - - **histories: List<History>**: Menyimpan kumpulan data log riwayat aktivitas perubahan sistem.
- Metode (*Methods*):
 - + **createTask(data: Task): Task**: Menambahkan objek tugas baru ke dalam daftar manajemen sistem.
 - + **updateTask(id: String, data: Task): Task**: Memperbarui data dari tugas tertentu berdasarkan parameter ID yang sesuai.
 - + **deleteTask(id: String): void**: Menghapus objek tugas tertentu dari sistem menggunakan parameter ID.
 - + **getTask(id: String): Task**: Mencari dan mengembalikan satu objek tugas spesifik berdasarkan ID.
 - + **getAllTasks(): List<Task>**: Mengambil dan mengembalikan seluruh daftar tugas yang tersimpan.
 - + **addHistory(task: Task, action: String, user: User): void**: Memicu pencatatan entri riwayat aktivitas baru yang dilakukan oleh pengguna terhadap tugas tertentu.
 - + **getAllHistoryByTask(taskId: String): List<History>**: Mengambil kumpulan daftar log riwayat khusus untuk satu tugas tertentu.
 - + **filterByPriority(priority: String): List<Task>**: Implementasi metode *interface* Filterable untuk menyaring tugas berdasarkan prioritas.
 - + **filterByStatus(status: String): List<Task>**: Implementasi metode *interface* Filterable untuk menyaring tugas berdasarkan kolom status.

- + **filterByLabel(label: Label): List<Task>** : Implementasi metode *interface* Filterable untuk menyaring tugas berdasarkan label.

3.3.2.4. Kelas TASK

Kelas dasar (*superclass*) yang memuat seluruh atribut dan fungsionalitas fundamental dari sebuah entitas kartu tugas. Kelas ini mengimplementasikan *interface* Trackable.

- Atribut (*Attributes*):
 - - **id: String**: Pengidentifikasi unik dari setiap tugas.
 - - **judul: String**: Nama atau judul utama dari tugas.
 - - **deskripsi: String**: Penjelasan mendetail mengenai instruksi atau konten tugas.
 - - **deadline: Date**: Batas tenggat waktu pengumpulan atau penyelesaian tugas.
 - - **status: String**: Status progres terkini (*To Do / In Progress / Review / Done*).
 - - **prioritas: String**: Tingkat kepentingan tugas (*Low / Medium / High*).
 - - **createdAt: Date**: Stempel waktu otomatis dari sistem saat tugas dibuat.
 - - **labels: List<Label>**: Representasi relasi agregasi yang menampung daftar objek label penyerta tugas.
- Metode (*Methods*):
 - + **changeStatus(status: String): void**: Mengubah nilai string status internal tugas.
 - + **setDeadline(date: Date): void**: Mengatur atau mendefinisikan ulang batas tenggat waktu tugas.
 - + **setPriority(priority: String): void**: Mengubah tingkatan prioritas tugas.
 - + **updateDeskripsi(desc: String): void**: Memperbarui isi deskripsi atau catatan teks tugas.
 - + **addLabel(label: Label): void**: Menyematkan objek label baru ke dalam daftar labels milik tugas.
 - + **removeLabel(label: Label): void**: Menghapus label tertentu yang sudah terpasang dari daftar tugas.
 - + **updateStatus(status: String): void**: Implementasi metode *interface* Trackable untuk memperbarui status progres tugas.
 - + **getStatus(): String**: Implementasi metode *interface* Trackable untuk mengambil status terkini tugas.

- + **getType(): String**: Fungsi polimorfisme dasar untuk mendapatkan klasifikasi jenis sub-class tugas.

3.3.2.5. Kelas USER

Kelas yang merepresentasikan entitas pengguna aplikasi (mahasiswa) yang mengelola tugas, menerima notifikasi, dan tercatat dalam log aktivitas.

- Atribut (*Attributes*):
 - - **id: String**: Pengidentifikasi unik untuk akun pengguna.
 - - **nama: String**: Nama lengkap dari pengguna aplikasi.
 - - **email: String**: Alamat surel aktif yang digunakan sebagai identitas akun.
 - - **password: String**: Kata sandi akun untuk kepentingan keamanan autentikasi.
 - - **role: String**: Peran spesifik pengguna di dalam sistem (contoh: Mahasiswa/Admin).
- Metode (*Methods*):
 - + **login(): void**: Menangani proses masuk pengguna dan memvalidasi kredensial ke sistem.
 - + **logout(): void**: Menangani proses keluar dari aplikasi dan menghancurkan sesi aktif.
 - + **register(): void**: Memproses pendaftaran akun baru dengan menyimpan nama, email, password, dan peran.
 - + **getProfile(): String**: Mengembalikan informasi profil pengguna yang sedang aktif untuk kebutuhan antarmuka.

3.3.2.6. Kelas PersonalTask

Kelas spesifik turunan dari TASK (*Inheritance*) yang merepresentasikan kartu tugas mandiri milik individu pengguna.

- Atribut (*Attributes*):
 - - **owner: User**: Objek pengguna (User) tunggal yang bertindak sebagai pemilik penuh atas tugas personal tersebut.
- Metode (*Methods*):

- + **getType(): String:** Melakukan *override* metode dari superclass untuk mengembalikan nilai string "PersonalTask".

3.3.2.7. Kelas GroupTask

Kelas spesifik turunan dari TASK (*Inheritance*) yang merepresentasikan kartu tugas berbasis kelompok atau proyek bersama.

- Atribut (*Attributes*):
 - - **member: List<User>:** Kumpulan objek pengguna yang terdaftar sebagai anggota dalam pengerjaan proyek.
 - - **assignedTo: User:** Objek pengguna spesifik yang ditunjuk sebagai penanggung jawab utama (*PIC*) dari tugas tersebut.
- Metode (*Methods*):
 - + **assignTo(user: User): void:** Mendelegasikan atau menetapkan tugas kartu tersebut kepada seorang anggota tertentu.
 - + **addMember(user: User): void:** Memasukkan objek pengguna baru ke dalam daftar keanggotaan kelompok proyek.
 - + **getType(): String:** Melakukan *override* metode dari superclass untuk mengembalikan nilai string "GroupTask".

3.3.2.8. Kelas DeadlineTask

Kelas spesifik turunan dari TASK (*Inheritance*) yang memfokuskan fungsionalitasnya pada pelacakan ketat batas jatuh tempo.

- Atribut (*Attributes*):
 - - **isOverdue: Boolean:** Indikator biner untuk menandai apakah tugas telah melewati batas tanggal tenggat waktu (*true*) atau belum (*false*).
- Metode (*Methods*):
 - + **checkOverdue(): Boolean:** Logika komparasi untuk memeriksa waktu sistem saat ini terhadap nilai atribut deadline guna memperbarui status keterlambatan.

- + **getType(): String:** Melakukan *override* metode dari superclass untuk mengembalikan nilai string "DeadlineTask".

3.3.2.9. Kelas Label

Kelas pendukung yang merepresentasikan entitas penanda visual atau kategori pengelompokan yang dapat disematkan ke dalam tugas.

- Atribut (*Attributes*):
 - - **id: String:** Pengidentifikasi unik untuk data setiap objek label.
 - - **nama: String:** Nama teks penanda dari label (contoh: "Tugas Kuliah", "Web Dev").
 - - **warna: String:** Nilai string kode warna visual untuk kebutuhan representasi di papan Kanban.
- Metode (*Methods*):
 - + **getNama(): String:** Mengambil dan mengembalikan nilai dari atribut nama label terkait.

3.3.2.10. Kelas History

Kelas pendukung yang merepresentasikan satu baris entri data log rekam jejak aktivitas atau perubahan yang terjadi di dalam aplikasi.

- Atribut (*Attributes*):
 - - **id: String:** Pengidentifikasi unik dari data entri log riwayat.
 - - **task: Task:** Referensi objek tugas terkait yang mengalami perubahan aktivitas.
 - - **action: String:** Deskripsi narasi mengenai jenis tindakan aktivitas yang terjadi (contoh: "Mengubah status ke Done").
 - - **historyAt: Date:** Stempel tanggal dan waktu kapan aktivitas perubahan tersebut dieksekusi.
 - - **historyBy: User:** Referensi objek pengguna yang melakukan tindakan perubahan sistem tersebut.
- Metode (*Methods*):

- + **historyTask(t: Task): void:** Menghubungkan dan mengeset relasi objek tugas yang akan dicatat riwayatnya.
- + **sortingHistory(t: Task): void:** Mengurutkan kronologi daftar log riwayat tugas berdasarkan stempel waktu aktivitas teranyar.
- + **getHistoryList(): List<Task> (atau List<History>):** Mengembalikan kumpulan data log riwayat aktivitas yang tercatat dalam sistem.

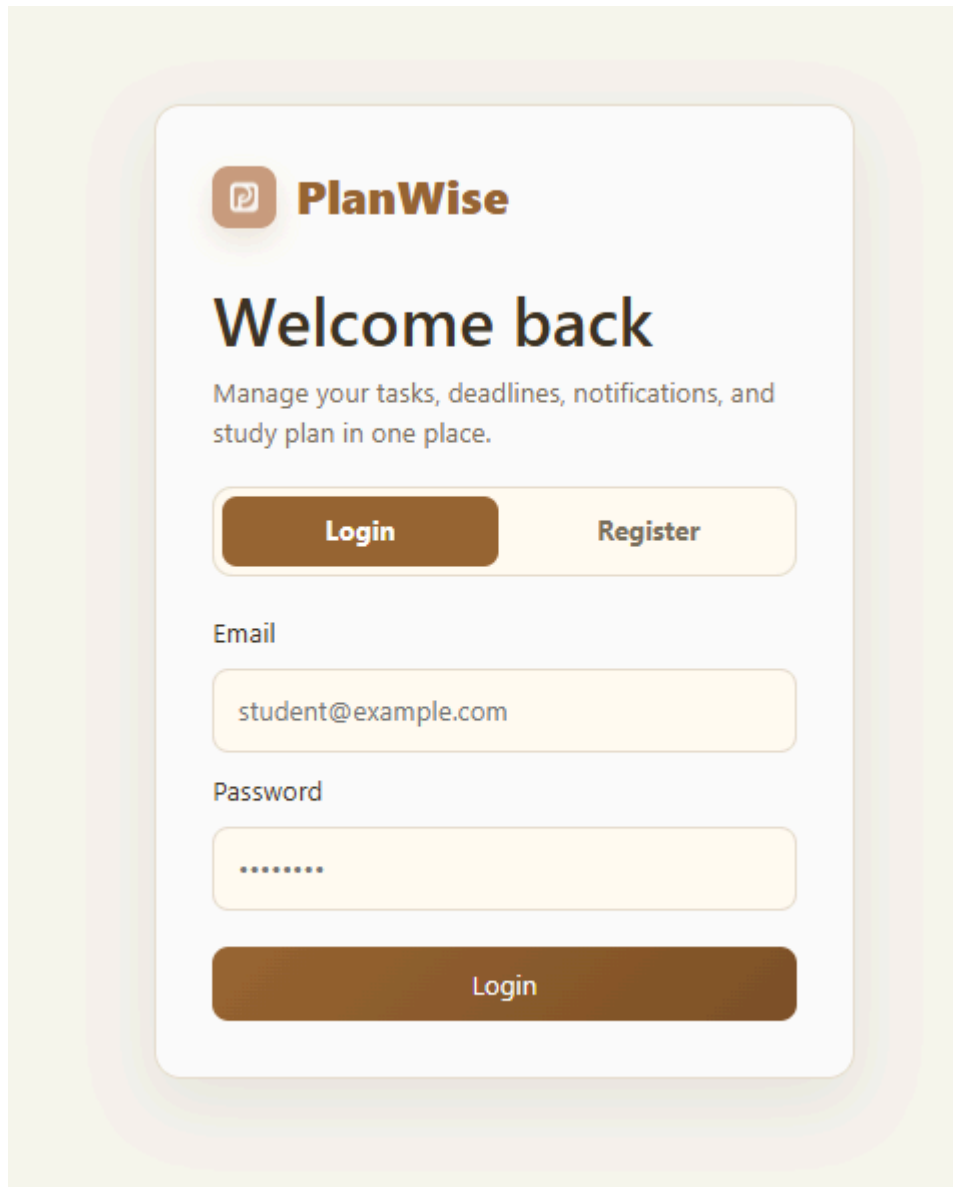
3.3.2.11. Kelas Notification

Kelas pendukung yang mengatur struktur mekanis pembuatan serta pengiriman pesan peringatan/informasi internal di dalam aplikasi (*in-app notification*).

- Atribut (*Attributes*):
 - - **id: String:** Pengidentifikasi unik data objek notifikasi.
 - - **pesan: String:** Isi teks pesan informasi notifikasi yang ditujukan kepada pengguna.
 - - **task: Task:** Referensi objek tugas yang menjadi pemicu lahirnya notifikasi (misal saat status/deadline berubah).
 - - **user: User:** Target objek pengguna yang ditunjuk untuk menerima kiriman notifikasi.
 - - **isRead: Boolean:** Status penanda apakah notifikasi sudah dibaca oleh pengguna (*true*) atau belum (*false*).
- Metode (*Methods*):
 - + **send(): void:** Mengirimkan dan memicu pemunculan data objek notifikasi baru ke akun pengguna target.
 - + **markAsRead(): void:** Mengubah nilai boolean atribut status *isRead* menjadi *true* setelah pengguna membaca notifikasi.
 - + **getMessage(): String:** Mengambil dan mengembalikan isi teks narasi dari atribut pesan.

Implementasi Antarmuka Pengguna (*User Interface*)

3.3.3 Halaman *Login*



The image shows a login and registration form for 'PlanWise'. The form is centered on a light green background with a subtle shadow. It features a logo with a square icon containing a stylized 'P' and the text 'PlanWise'. Below the logo is a large heading 'Welcome back' and a subtext 'Manage your tasks, deadlines, notifications, and study plan in one place.' There are two buttons: a dark brown 'Login' button and a light yellow 'Register' button. Below these are input fields for 'Email' (containing 'student@example.com') and 'Password' (containing seven dots). A final dark brown 'Login' button is at the bottom.

 **PlanWise**

Welcome back

Manage your tasks, deadlines, notifications, and study plan in one place.

Login **Register**

Email

student@example.com

Password

.....

Login

3.3.4 Halaman Registrasi


PlanWise

Create account

Manage your tasks, deadlines, notifications, and study plan in one place.

Login

Register

Nama

Nama lengkap

Email

student@example.com

Password

Minimal 6 karakter

Register

3.3.5 Halaman Dashboard



Dashboard
My Tasks
Board
Groups
Calendar
Labels
History

FAVORITES

- Web Development
- Database Project
- Study Plan

Stay organized.
Plan your tasks and track your progress every day.

Search tasks, descriptions, labels...
Ctrl + K

ihsan
ihsan@gmail.com
Logout

Selasa, 16 Juni 2026
2 notifikasi belum dibaca

Welcome back, ihsan!
Kamu punya 0 task, 0 sedang dikerjakan, dan 0 selesai.

Total Tasks
0

In Progress
0

Completed
0

High Priority
0

Recent Tasks
Belum ada task.

Open Board

Task Progress

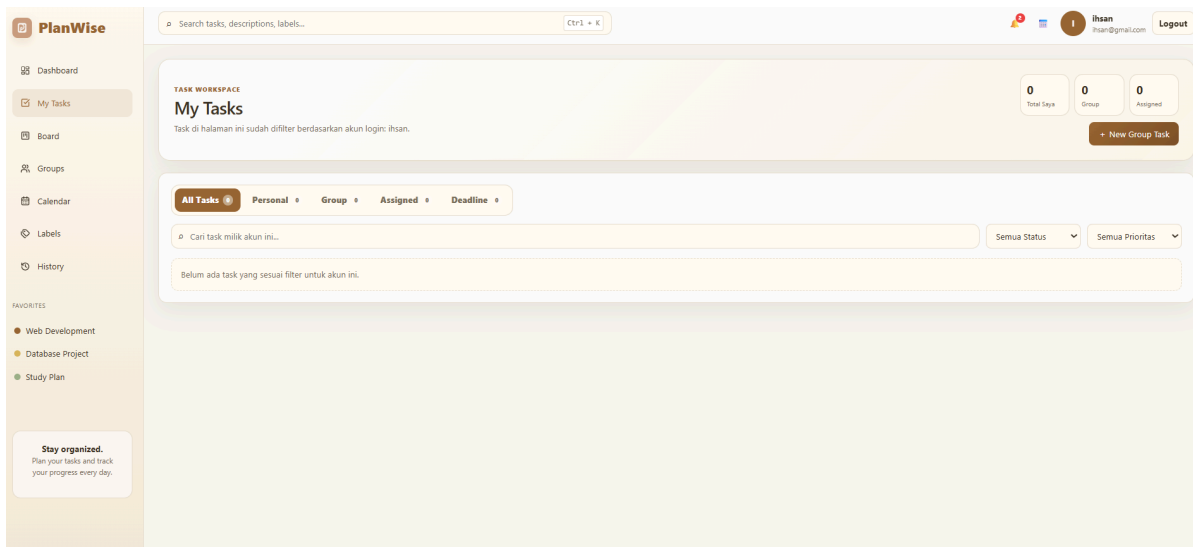
0%
Completed

Status diubah dari IN_PROGRESS ke TODO
16/6/2026, 21:58:43
Status diubah dari TODO ke IN_PROGRESS
16/6/2026, 21:58:42
Status diubah dari REVIEW ke TODO
16/6/2026, 21:27:09
Status diubah dari TODO ke REVIEW
16/6/2026, 21:27:08

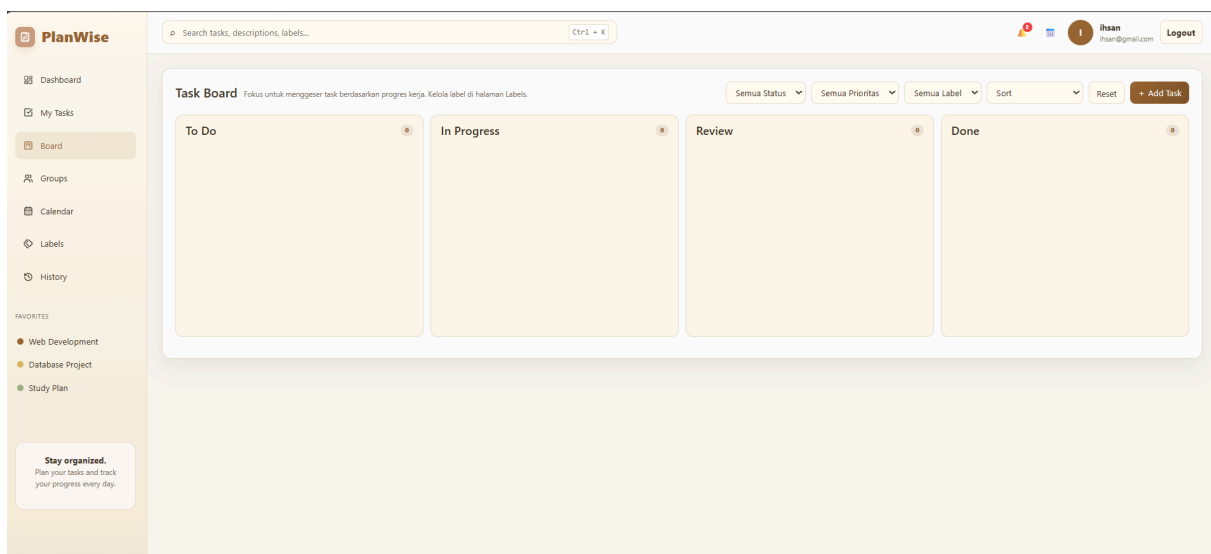
Upcoming Deadlines
Belum ada deadline aktif.

0 overdue

31

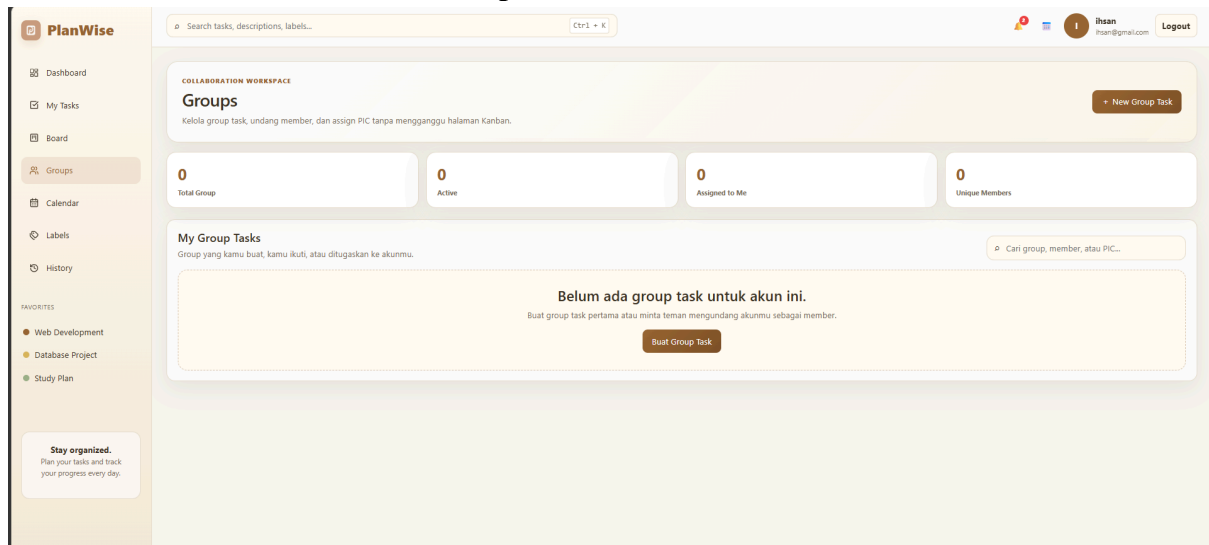


3.3.6 Halaman Tasks

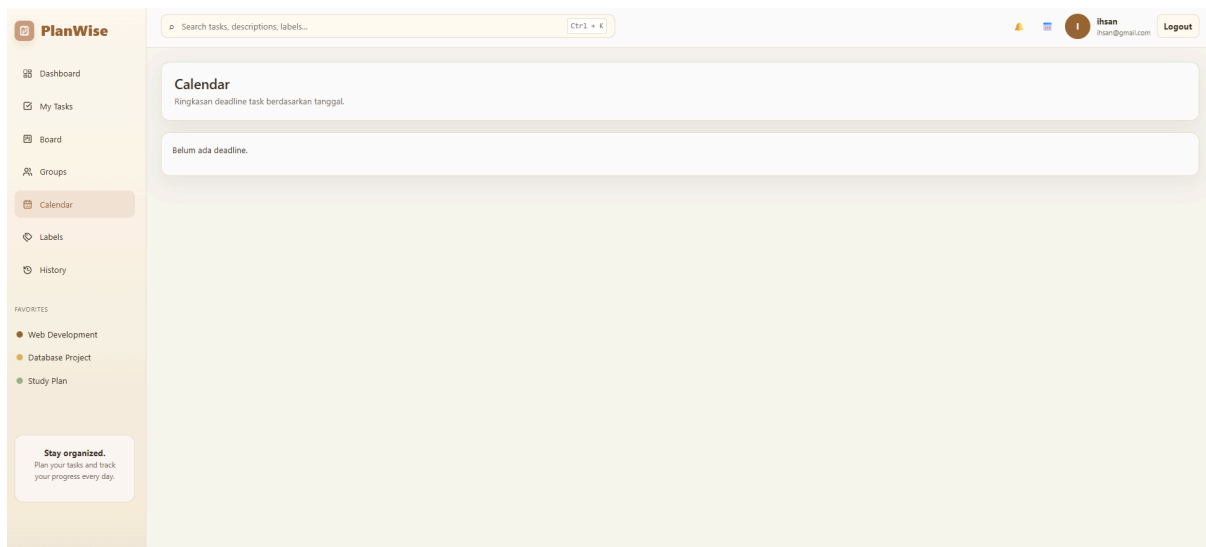


3.3.7 Halaman Board

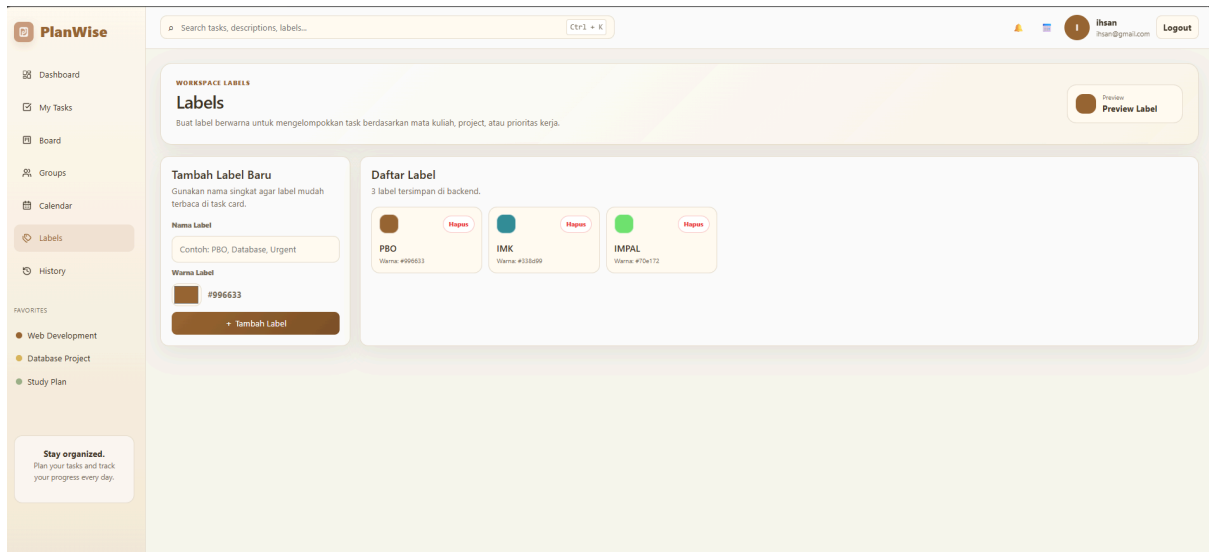
3.3.8 Halaman Group



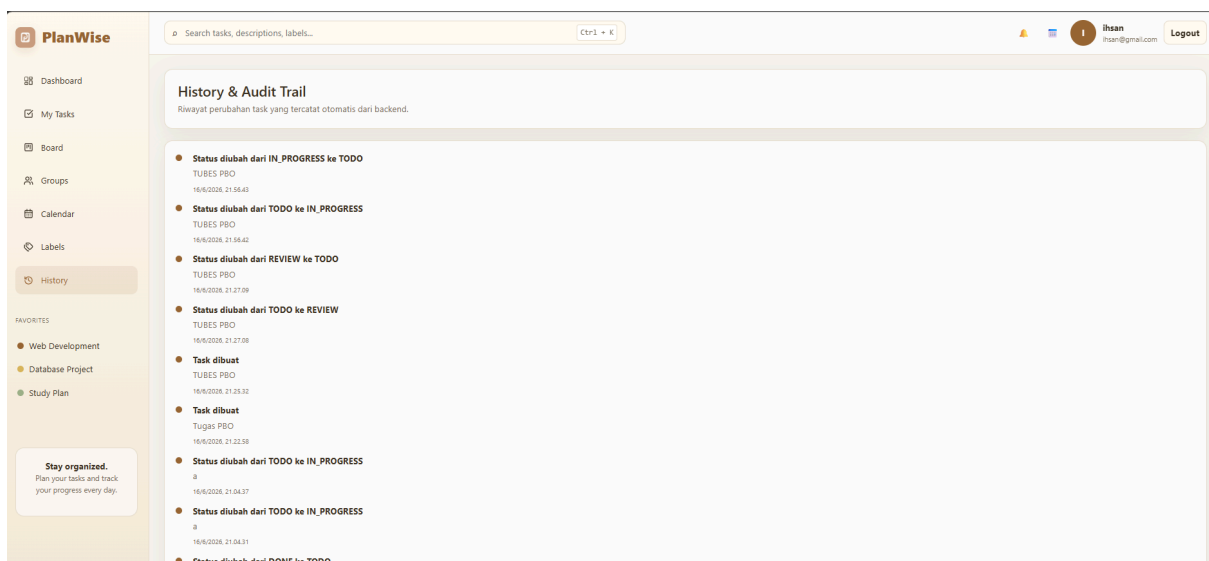
3.3.9 Halaman Calender



3.3.10 Halaman Label



3.3.11 Halaman History



4 Pengujian Sistem

4.1 Tujuan Pengujian

Pengujian sistem PlanWise bertujuan untuk memverifikasi bahwa seluruh fitur yang telah diimplementasikan berjalan sesuai dengan spesifikasi kebutuhan fungsional yang telah ditetapkan, serta memastikan tidak ada kesalahan logika (bug) pada alur utama aplikasi.

4.2 Lingkungan Pengujian

4.2.1 Perangkat Lunak Pengujian

Sistem ini diujikan dengan spesifikasi perangkat lunak sebagai berikut;

- Sistem Operasi: Windows 11
- Bahasa Pemrograman Backend: Java 17
- Framework Backend: Spring Boot 3.5
- Database: PostgreSQL (Supabase)
- Framework Frontend: React + Vite
- Browser: Google Chrome (versi terbaru)
- Tools Pengujian API: Postman / browser langsung

4.2.2 Perangkat Keras Pengujian

Sistem ini diujikan dengan spesifikasi perangkat Keras sebagai berikut;

- Prosesor: Intel Core i5 / AMD Ryzen 5
- Memori (RAM): 8 GB
- Penyimpanan: SSD 256 GB
- Koneksi: Internet (untuk akses Supabase cloud)

4.3 Metode Pengujian

4.3.1 (Black Box/White Box) Testing

Pengujian dilakukan dengan metode *Black Box*, yaitu menguji fungsionalitas sistem berdasarkan input dan output yang diharapkan tanpa melihat detail implementasi internal kode. Penguji berfokus pada apakah setiap fitur menghasilkan respons yang benar untuk setiap skenario.

4.3.2 Skenario Pengujian

Pengujian dilakukan pada seluruh fitur utama sistem. Skenario pengujian dapat dilihat sebagai berikut

Tabel 2 Identifikasi Pengujian

Kelas Uji	Butir Uji	Identifikasi	Jenis Pengujian	Penguji
Login	Pengujian fungsi login dengan data valid	AO-LI01	Black Box	Hakim Akram Azizi
Login	Pengujian login dengan data tidak valid	AO-LI02	Black Box	Hakim Akram Azizi
Logout	Pengujian fungsi logout	AO-LO01	Black Box	Hakim Akram Azizi
Manajemen Task	Menambahkan task baru	AO-TASK01	Black Box	Hakim Akram Azizi
Manajemen Task	Menambah task tanpa judul	AO-TASK02	Black Box	Hakim Akram Azizi

Manajemen Task	Mengubah data task	AO-TASK03	Black Box	Hakim Akram Azizi
Manajemen Task	Menghapus task	AO-TASK04	Black Box	Hakim Akram Azizi
Manajemen Task	Mengubah status task	AO-TASK05	Black Box	Hakim Akram Azizi
Label & Filter	Membuat label baru	AO-LABEL01	Black Box	Hakim Akram Azizi
Label & Filter	Menambahkan label ke task	AO-LABEL02	Black Box	Hakim Akram Azizi
Label & Filter	Filter berdasarkan label	AO-LABEL03	Black Box	Hakim Akram Azizi
Label & Filter	Filter berdasarkan prioritas	AO-LABEL04	Black Box	Hakim Akram Azizi
Label & Filter	Sort berdasarkan deadline	AO-LABEL05	Black Box	Hakim Akram Azizi
History	Riwayat tercatat saat task dibuat	AO-HIST01	Black Box	Hakim Akram Azizi
History	Riwayat tercatat saat status diubah	AO-HIST02	Black Box	Hakim Akram Azizi
Notification	Notifikasi saat task dibuat	AO-NOTIF01	Black Box	Hakim Akram Azizi
Notification	Menandai notifikasi sudah dibaca	AO-NOTIF02	Black Box	Hakim Akram Azizi

5 Hasil Uji

Hasil uji berdasarkan skenario yang telah dibuat pada bab sebelumnya.

5.1 Fitur *Login*

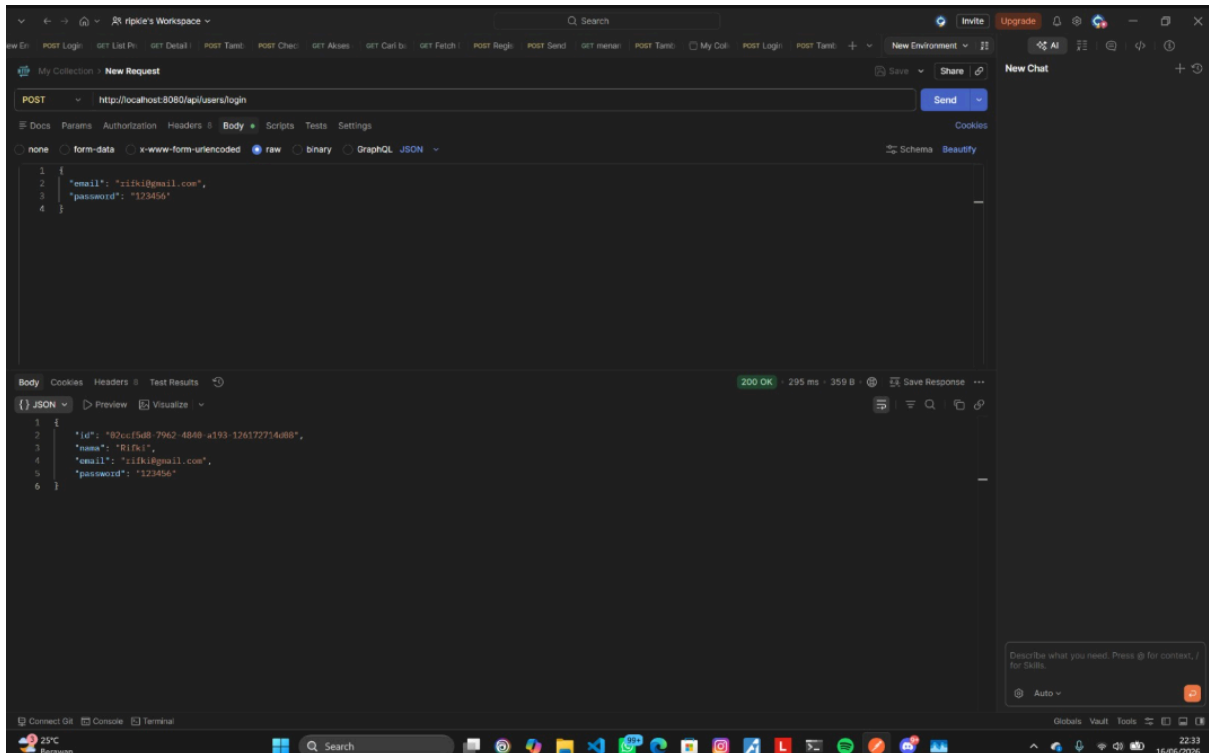
5.1.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 3 Kasus Uji Fitur Login

Identifikasi	Skenario
AO-LI01	Berhasil <i>Login</i>
AO-LI02	<i>Username</i> Salah
AO-LI03	<i>Password</i> Salah
AO-LI04	<i>Field Kosong</i>
AO-LI05	<i>Logout</i>

5.1.2 Hasil



Hasil Uji Sebagai Berikut

Tabel 4 Hasil Uji Fitur Login

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-LI01	<i>Username</i> : admin <i>Password</i> : Admin123	<i>Login</i> Berhasil	Sesuai	Pengujian berhasil
AO-LI02	<i>Username</i> : salah <i>Password</i> : Admin123	Pesan <i>Error</i>	Sesuai	Pengujian berhasil
AO-LI02	<i>Username</i> : admin <i>Password</i> : salah	Pesan <i>Error</i>	Sesuai	Pengujian berhasil

AO-LI02	Kosong	Validasi muncul	Sesuai	Pengujian berhasil
---------	--------	-----------------	--------	--------------------

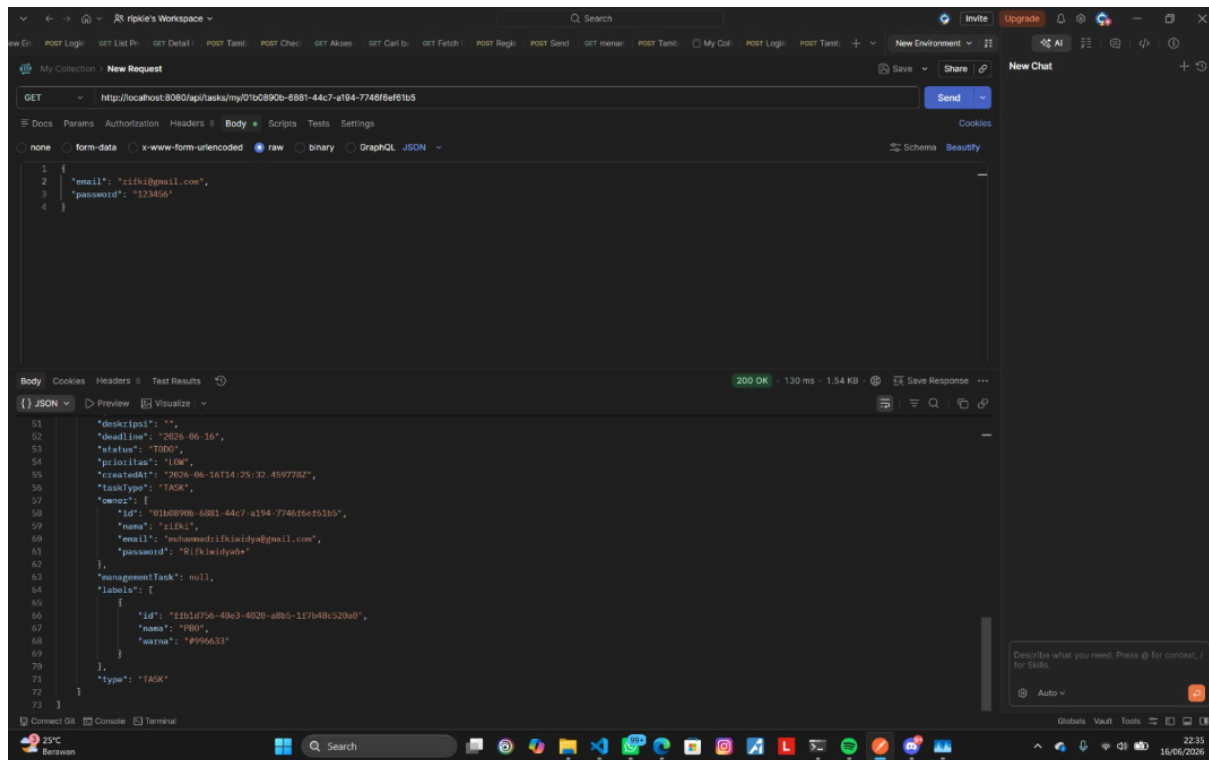
5.3 Fitur Manajemen Task

5.3.1 Kasus Uji

Tabel 7 Kasus Uji Fitur Manajemen Task

Identifikasi	Skenario
AO-TASK01	Menambahkan task baru
AO-TASK02	Menambahkan task tanpa judul
AO-TASK03	Mengubah data task
AO-TASK04	Menghapus task
AO-TASK05	Mengubah status task

5.1.3 Hasil



Hasil Uji Sebagai Berikut

Tabel 6 Hasil Uji Fitur Manajemen Task

Identifikasi	Input	Hasil yang Diharapkan	Hasil Sebenarnya	Status
AO-TASK01	Mengisi seluruh field task dan klik Save	Task berhasil dibuat	Sesuai	Berhasil
AO-TASK02	Menyimpan task tanpa judul	Muncul validasi bahwa judul wajib diisi	Sesuai	Berhasil
AO-TASK03	Mengubah deskripsi atau deadline task	Data task diperbarui	Sesuai	Berhasil
AO-TASK04	Menekan tombol Delete	Task terhapus dari board	Sesuai	Berhasil
AO-TASK05	Mengubah status To Do → In Progress	Status task berubah pada board	Sesuai	Berhasil

5.2 Fitur Label & Filter

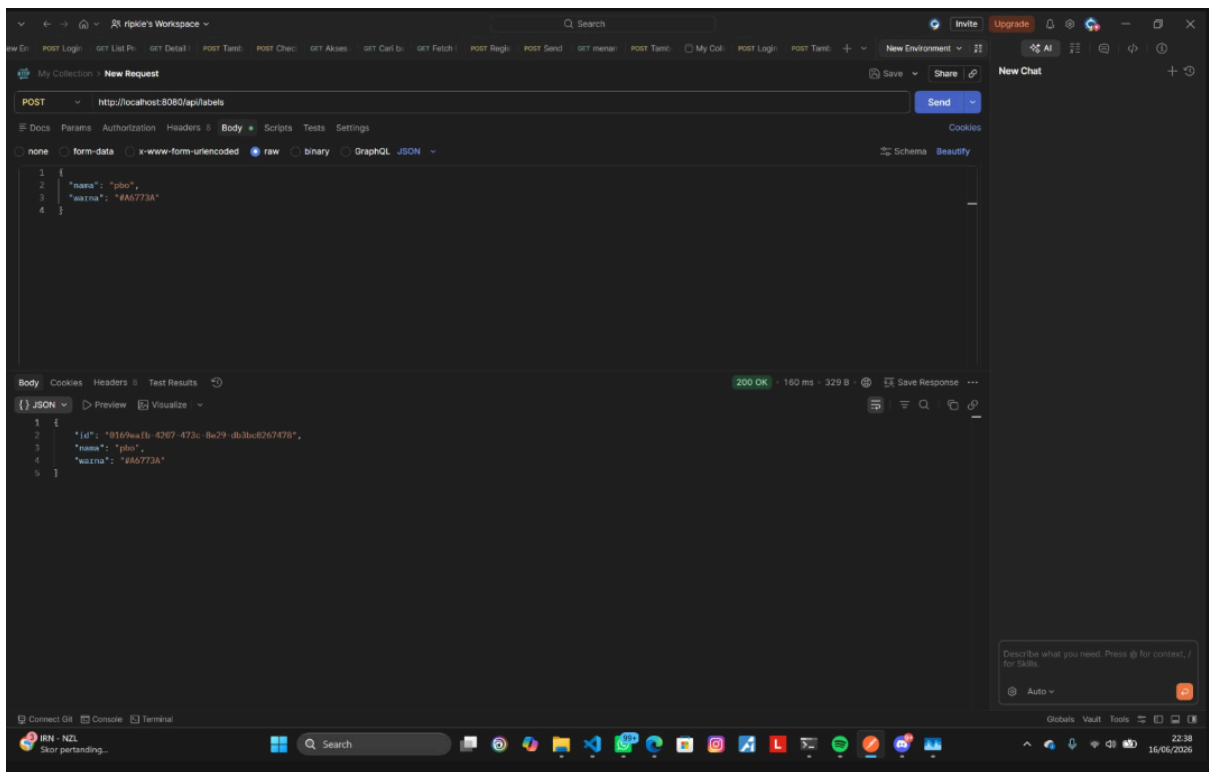
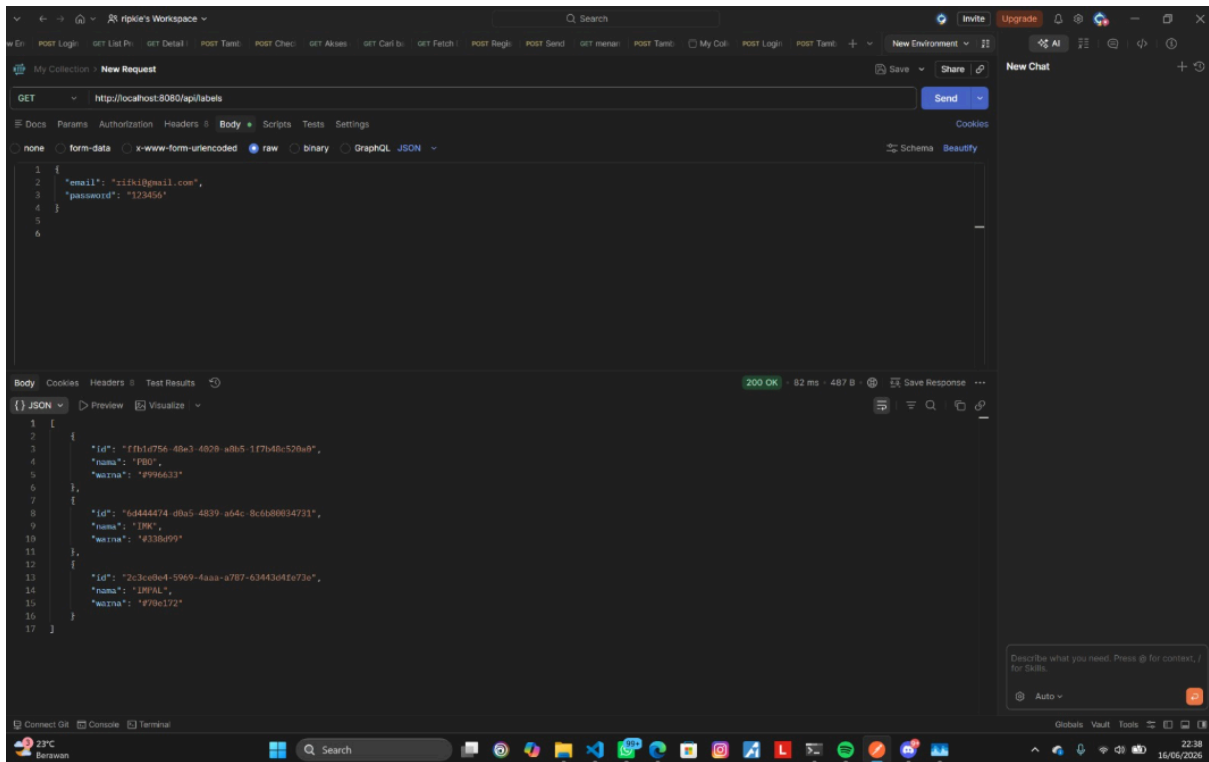
5.2.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut

Tabel 7 Kasus Uji Fitur Label & Filter

Identifikasi	Skenario
AO-LABEL01	Membuat label baru
AO-LABEL02	Menambahkan label ke task
AO-LABEL03	Filter berdasarkan label
AO-LABEL04	Filter berdasarkan prioritas
AO-LABEL05	Sort berdasarkan deadline

5.2.2 Hasil



Hasil Uji Sebagai Berikut

Tabel 8 Hasil Uji Fitur Label & Filter

Identifikasi	Input	Hasil yang Diharapkan	Hasil Sebenarnya	Status

AO-LABEL01	Membuat label "Kuliah"	Label tersimpan	Sesuai	Berhasil
AO-LABEL02	Menambahkan label pada task	Label tampil pada task	Sesuai	Berhasil
AO-LABEL03	Filter label "Kuliah"	Hanya task berlabel Kuliah yang tampil	Sesuai	Berhasil
AO-LABEL04	Filter prioritas High	Hanya task prioritas High yang tampil	Sesuai	Berhasil
AO-LABEL05	Sort deadline	Task diurutkan berdasarkan deadline terdekat	Sesuai	Berhasil

5.3 Fitur History

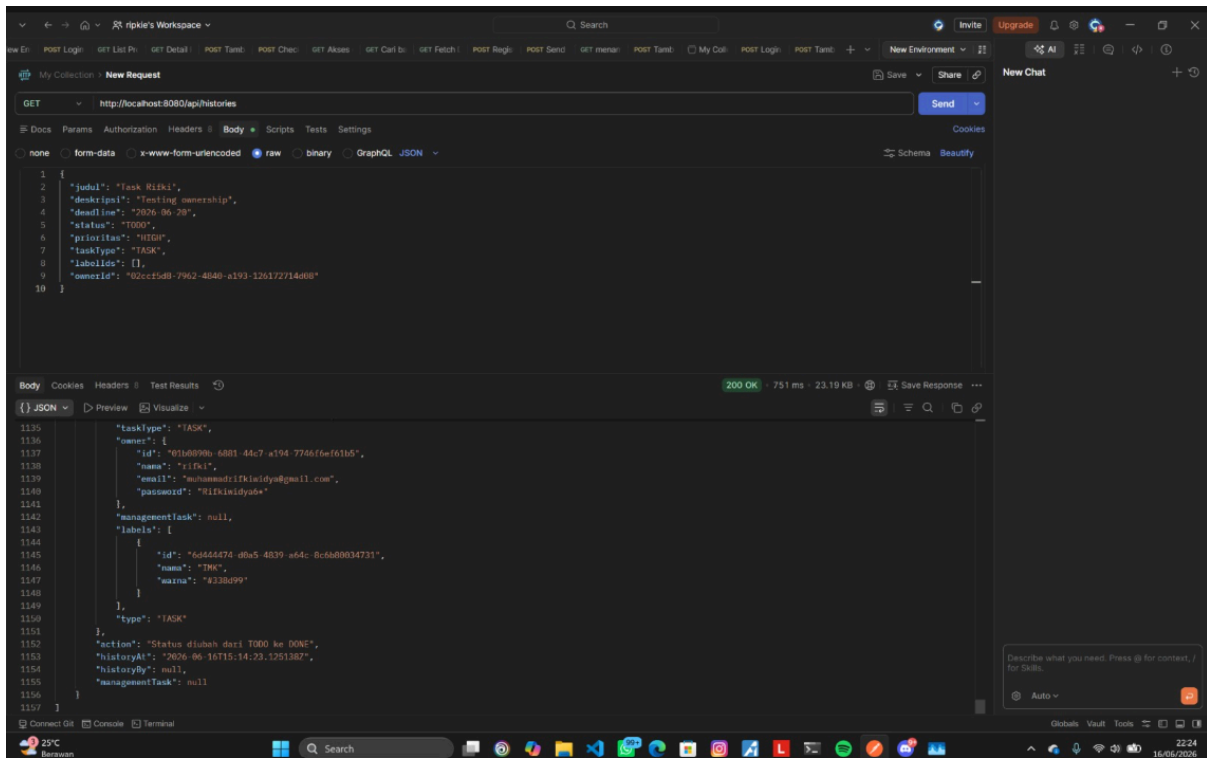
5.3.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut

Tabel 9 Kasus Uji Fitur History

Identifikasi	Skenario
AO-HIST01	Cek History Setelah Membuat Task Baru
AO-HIST02	Cek History Setelah Mengupdate Status

5.3.2 Hasil



Hasil Uji Sebagai Berikut

Tabel 10 Hasil Uji Fitur History

Identifikasi	Input	Hasil yang Diharapkan	Hasil Sebenarnya	Status
AO-HIST01	Membuat task baru	Log aktivitas tercatat	Sesuai	Berhasil
AO-HIST02	Mengubah status task	Riwayat perubahan tersimpan	Sesuai	Berhasil

5.4 Fitur Notifikasi

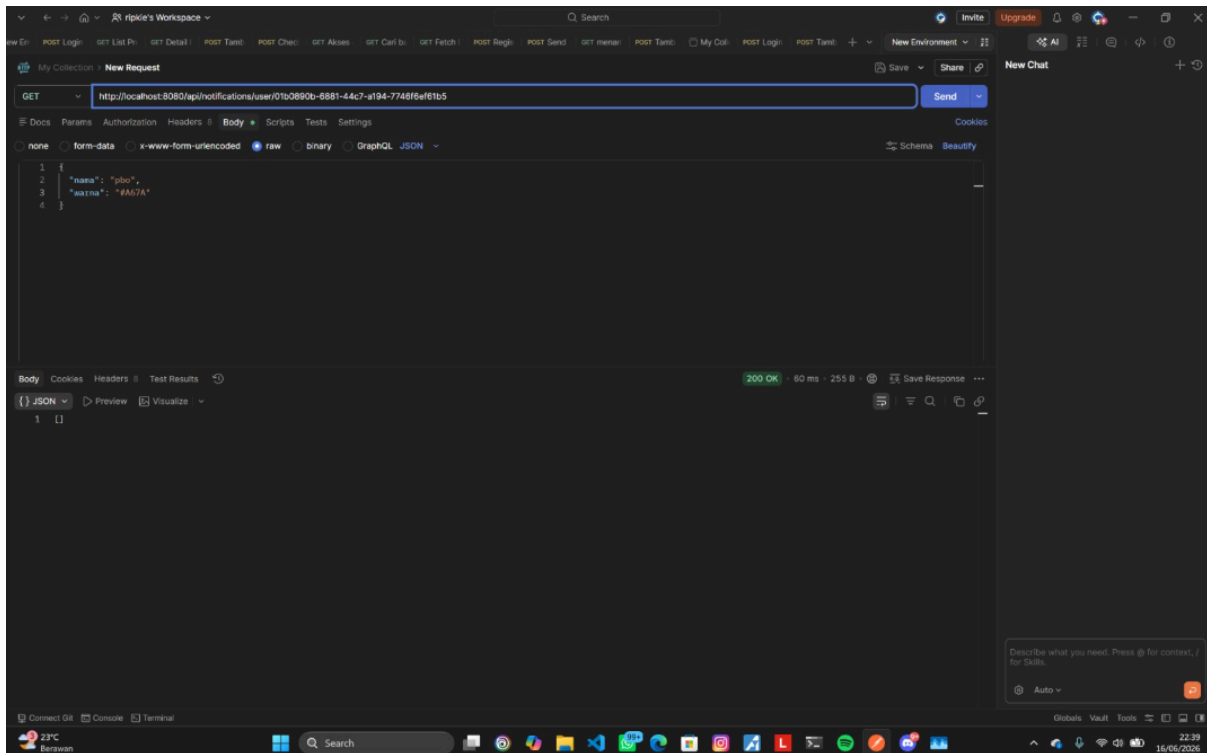
5.4.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut

Tabel 9 Kasus Uji Fitur History

Identifikasi	Skenario
AO-NOTIF01	Cek Notifikasi Setelah Task Dibuat
AO-NOTIF02	Menandai Notif Sudah Dibaca

5.4.2 Hasil



Hasil Uji Sebagai Berikut

Tabel 10 Hasil Uji Fitur History

Identifikasi	Input	Hasil yang Diharapkan	Hasil Sebenarnya	Status
AO-NOTIF01	Membuat task baru	Notifikasi muncul pada panel notifikasi	Sesuai	Berhasil
AO-NOTIF02	Klik tombol Mark as Read	Status notifikasi berubah menjadi dibaca	Sesuai	Berhasil

6 Penutup

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, PlanWise berhasil dikembangkan sebagai aplikasi manajemen tugas berbasis web yang memenuhi seluruh tujuan yang telah ditetapkan. Kelima fitur utama (Autentikasi & Profil Pengguna, Manajemen Task Board/CRUD, Status Tracking & Kanban Board, Label/Prioritas/Filter/Sort, serta Riwayat Aktivitas & Notifikasi) telah diimplementasikan dan lulus pengujian Black Box.

Konsep Pemrograman Berorientasi Objek berhasil diterapkan secara konsisten: *encapsulation* diterapkan pada seluruh entity, *inheritance* diimplementasikan melalui hierarki Task → PersonalTask/GroupTask/DeadlineTask, *polymorphism* melalui override method getType(), serta *interface* Trackable dan Filterable sebagai kontrak abstraksi. Arsitektur backend

menggunakan Spring Boot dengan pola *layered architecture* (Controller → Service → Repository → Entity) menjadikan kode modular dan mudah dikembangkan.

Saran Pengembangan

Beberapa saran untuk pengembangan PlanWise ke depannya:

1. Autentikasi berbasis JWT — mengganti mekanisme session sederhana dengan token JWT untuk keamanan yang lebih baik.
2. Fitur real-time — menggunakan WebSocket untuk pembaruan kanban board secara langsung tanpa perlu refresh halaman.
3. Push Notification — mengintegrasikan notifikasi eksternal (email atau browser push notification) untuk pengingat deadline.
4. Versi mobile — mengembangkan aplikasi mobile menggunakan React Native atau Flutter untuk akses di perangkat Android/iOS.
5. Pagination & Search — menambahkan fitur pencarian full-text dan paginasi pada daftar task untuk performa lebih baik saat data bertambah besar.

DAFTAR PUSTAKA

- ATlassian. (N.D.). *TRELLO: KELOLA PROYEK TIM ANDA DARI MANA SAJA*. DIAKSES PADA 16 JUNI 2026, DARI [HTTPS://TRELLO.COM/](https://trello.com/)
- NOTION LABS, INC. (N.D.). *NOTION: ONE WORKSPACE. EVERY TEAM*. DIAKSES PADA 16 JUNI 2026, DARI [HTTPS://WWW.NOTION.SO/](https://www.notion.so/)
- REVOU. (2025, 3 FEBRUARI). TRELLO. DIAKSES PADA 16 JUNI 2026, DARI [HTTPS://WWW.REVOU.CO/KOSAKATA/TRELLO](https://www.revou.co/kosakata/trello)
- REVOU. (2025, 3 FEBRUARI). NOTION. DIAKSES PADA 16 JUNI 2026, DARI [HTTPS://WWW.REVOU.CO/KOSAKATA/NOTION](https://www.revou.co/kosakata/notion)

LAMPIRAN

Lampiran A Pembagian Tugas

NIM	Nama	Tugas	Kontribusi (100%)	Penyelesaian (%)
103012400009	Hakim Akram Azizi	Mengerjakan	20%	100%
103012400223	Azh Zhahir Ramadhan P	Mengerjakan	20%	100%
103012400354	Muhammad Ihsan Abdurrasyid	Mengerjakan	20%	100%
103012400186	Muhammad Rifki Widya	Mengerjakan	20%	100%
103012400132	Muhammad Sylva Sadjandegi	Mengerjakan	20%	100%

Lampiran B Source Code Penting

Class Task dipilih sebagai source code penting pada aplikasi PlanWise karena merupakan kelas inti yang menjadi dasar pengelolaan seluruh tugas dalam sistem. Kelas ini menyimpan atribut utama seperti judul, deskripsi, deadline, status, prioritas, dan label, serta menyediakan berbagai fungsi penting seperti perubahan status, pengaturan deadline, dan manajemen label. Selain itu, class Task juga menjadi superclass bagi PersonalTask, GroupTask, dan DeadlineTask, sehingga berperan sebagai pusat implementasi konsep Object-Oriented Programming (OOP) dan mendukung hampir seluruh fitur utama aplikasi PlanWise.

```

1  package com.planwise.backend.entity;
2
3  import com.planwise.backend.interfaces.Trackable;
4
5
6  import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
7  import jakarta.persistence.*;
8  import lombok.*;
9  import java.time.LocalDate;
10 import java.time.Instant;
11 import java.util.ArrayList;
12 import java.util.List;
13
14 @Entity
15 @Table(name = "tasks")
16 @Inheritance(strategy = InheritanceType.JOINED)
17 @Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
18 @JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
19 public class Task implements Trackable {
20
21     @Id
22     private String id;
23
24     @Column(nullable = false)
25     private String judul;
26
27     @Column(columnDefinition = "TEXT")
28     private String deskripsi;
29
30     private LocalDate deadline;
31
32     private String status;

```

jakarta.persistence.InheritanceType
Source: *jakarta.persistence-api-3.1.0.jar*

```

34     private String prioritas;
35
36     @Column(name = "created_at")
37     private Instant createdAt;
38
39     @Column(name = "task_type")
40     private String taskType;
41
42     @ManyToOne(fetch = FetchType.LAZY)
43     @JoinColumn(name = "owner_id")
44     @JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
45     private User owner;
46
47     @ManyToOne(fetch = FetchType.LAZY)
48     @JoinColumn(name = "management_task_id")
49     @JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
50     private ManagementTask managementTask;
51
52     @ManyToMany(fetch = FetchType.LAZY)
53     @JoinTable(
54         name = "task_labels",
55         joinColumns = @JoinColumn(name = "task_id"),
56         inverseJoinColumns = @JoinColumn(name = "label_id")
57     )
58     @Builder.Default
59     private List<Label> labels = new ArrayList<>();
60
61     public void changeStatus(String status) {
62         this.status = status;
63     }

```

```

65     public void setDeadline(LocalDate date) {
66         this.deadline = date;
67     }
68
69     public void setPriority(String priority) {
70         this.prioritas = priority;
71     }
72
73     public void updateDeskripsi(String desc) {
74         this.deskripsi = desc;
75     }
76
77     public void addLabel(Label label) {
78         if (labels == null) {
79             labels = new ArrayList<>();
80         }
81         boolean exists = labels.stream().anyMatch(item -> item.getId().equals(label.getId()));
82         if (!exists) {
83             labels.add(label);
84         }
85     }
86
87     public void removeLabel(Label label) {
88         if (labels != null) {
89             labels.removeIf(item -> item.getId().equals(label.getId()));
90         }
91     }
92

```

```

93     @Override
94     public void updateStatus(String status) {
95         this.status = status;
96     }
97
98     public String getType() {
99         return taskType;
100     }
101 }

```

Lampiran C Screenshot Sistem



Welcome back

Manage your tasks, deadlines, notifications, and study plan in one place.

Login

Register

Email

student@example.com

Password

.....

Login



Create account

Manage your tasks, deadlines, notifications, and study plan in one place.

Login

Register

Nama

Nama lengkap

Email

student@example.com

Password

Minimal 6 karakter

Register

PlanWise

Dashboard

My Tasks

Board

Groups

Calendar

Labels

History

FAVORITES

Web Development

Database Project

Study Plan

Stay organized.
Plan your tasks and track your progress every day.

Search tasks, descriptions, labels...
Ctrl + K

Welcome back, ihsan!

Kamu punya 0 task, 0 sedang dikerjakan, dan 0 selesai.

Selasa, 16 Juni 2026

2 notifikasi belum dibaca

Total Tasks

0

In Progress

0

Completed

0

High Priority

0

Recent Tasks

Belum ada task.

Open Board

Task Progress

View History

0% Completed

Status diubah dari IN_PROGRESS ke TODO
16/6/2026, 21:56:43

Status diubah dari TODO ke IN_PROGRESS
16/6/2026, 21:59:42

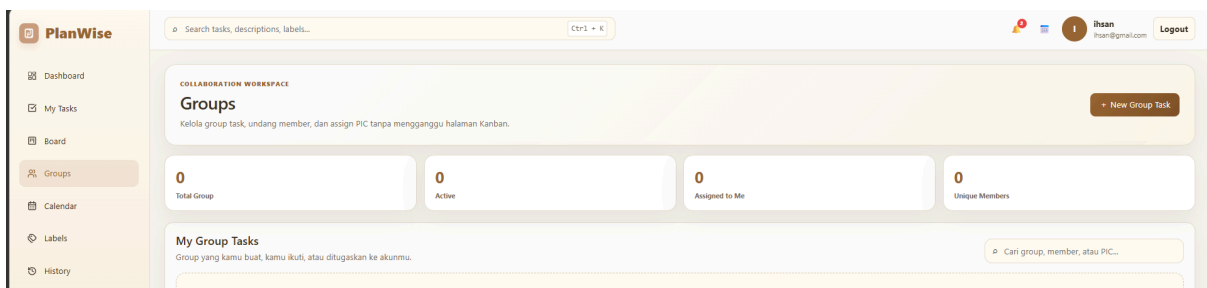
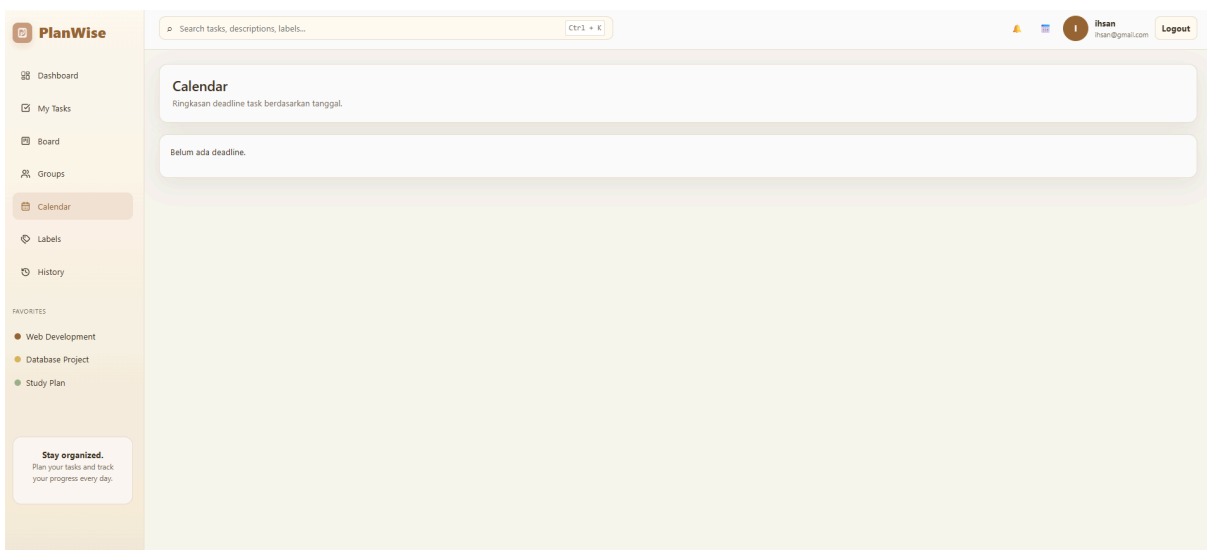
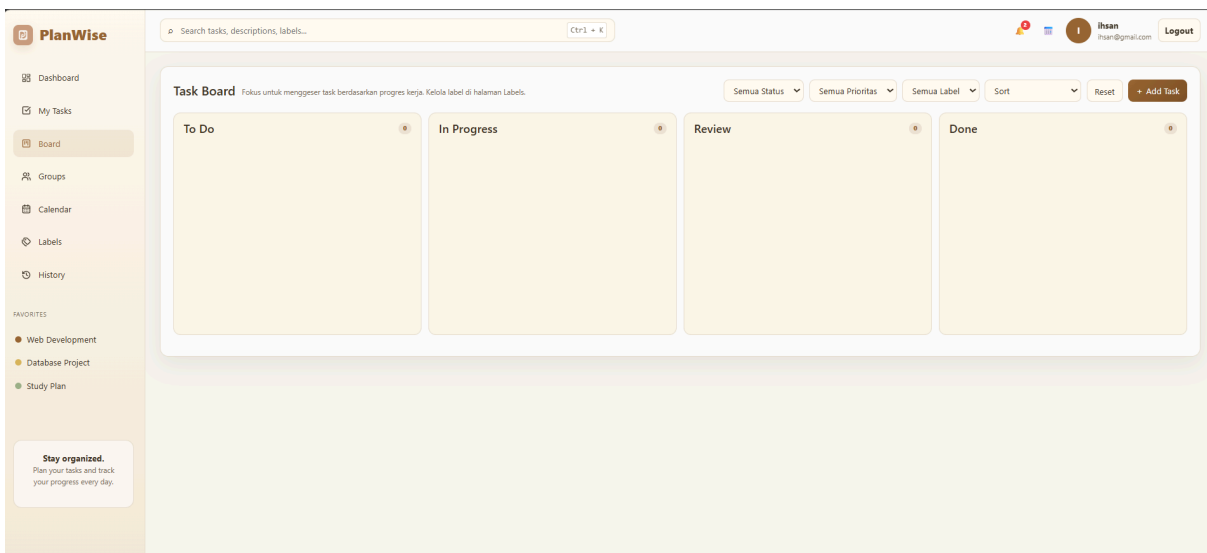
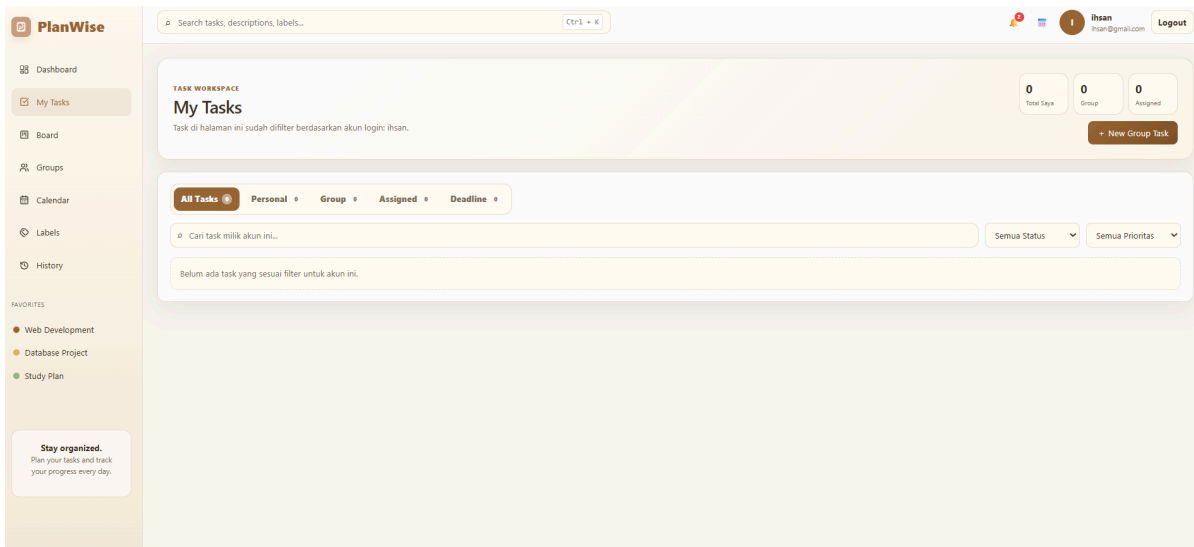
Status diubah dari REVIEW ke TODO
16/6/2026, 21:27:09

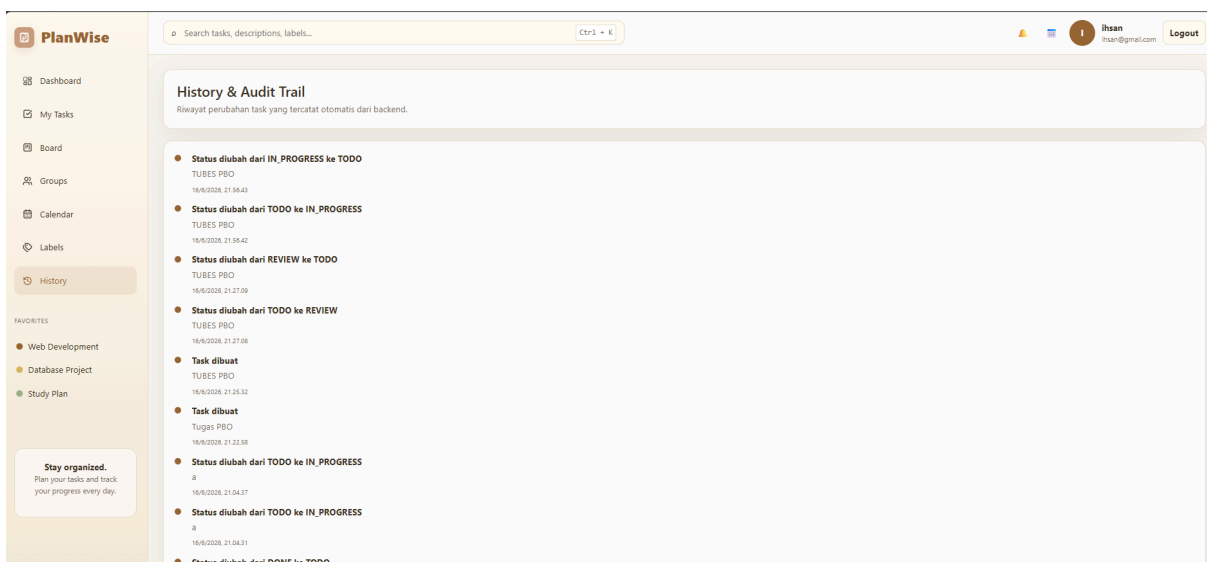
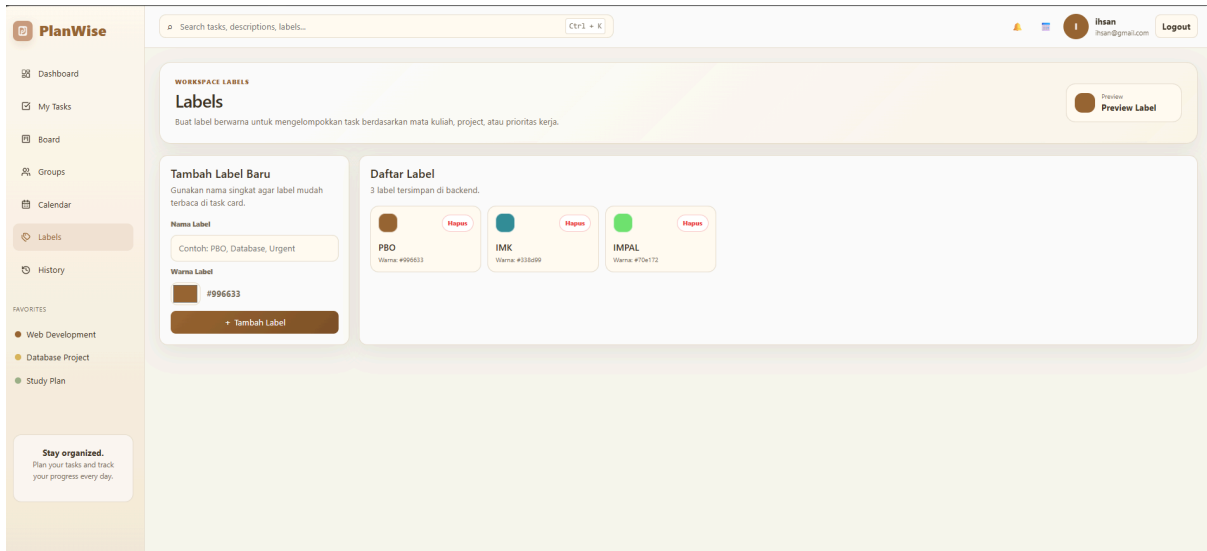
Status diubah dari TODO ke REVIEW
16/6/2026, 21:27:08

Upcoming Deadlines

Belum ada deadline aktif.

0 overdue





Lampiran D Manual Instalasi Sistem

 **Installation**

Clone Repository

```
git clone <repository-url>  
cd planwise-website
```



Backend

```
cd backend  
./mvnw spring-boot:run
```



Backend berjalan pada:

```
http://localhost:8080
```



Frontend

```
cd frontend  
npm install  
npm run dev
```



Frontend berjalan pada:

```
http://localhost:5173
```

